

EXPERIMENT 1

PROBLEM STATEMENT: Introduction and installation of WEKA tool.

THEORY:

Weka (Waikato Environment for Knowledge Analysis) is an open source suite for machine learning software written in Java, designed for data mining and data analysis tasks. Developed by the University of Waikato in New Zealand, it is widely used for research, education and industrial applications due to its user-friendly graphical interface (GUI) and comprehensive set of algorithms.

Steps of Installation:-

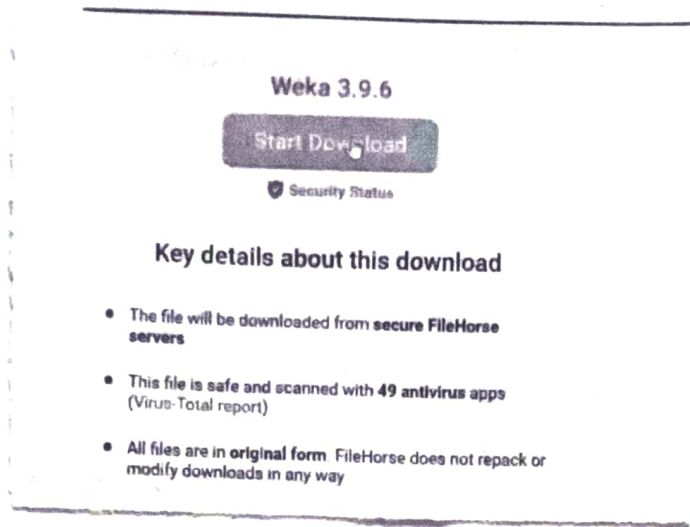
Step 1: Visit the website using web browser. Click on Free Download.

The screenshot shows the SourceForge page for 'Weka for PC'. On the left, there is an advertisement for 'CorelDRAW Graphics Suite 2025' with the tagline 'Accelerate your creativity with a more powerful Painterly Brush Tool.' The main content area features the 'Weka for PC' download button, a 'Free Download' button, and a 'Security Status' indicator. Below these, there is a table with tabs for 'Review', 'Screenshots', and 'Old Versions'. The 'Review' tab is active, showing details about the latest version (3.9.6), the reviewer (Daniel Leblanc), the operating system (Windows XP64 / Vista64 / Windows 7 64 / Windows 8 64 / Windows 10 64), the user rating (4 stars), and the author/product (Weka Team / External Link). A description of Weka as a machine learning software developed at the University of Waikato is also present. On the right side, there are sponsored links for NordVPN (Get 71% Off) and a 'Recommended' section listing Visual Studio, UltraEdit, dbForge Studio, and dbForge SQL.

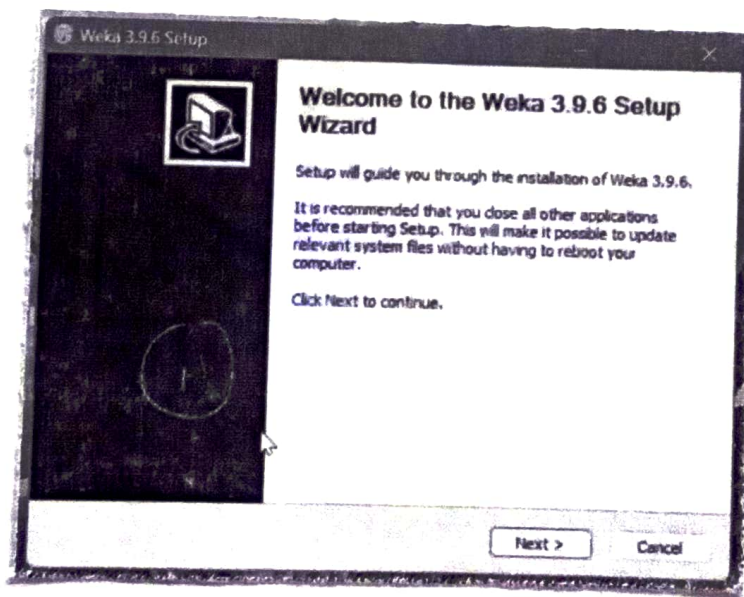
Review	Screenshots	Old Versions
Latest Version	Weka 3.9.6	
Review by	Daniel Leblanc	
Operating System	Windows XP64 / Vista64 / Windows 7 64 / Windows 8 64 / Windows 10 64	
User Rating	★★★★☆ Click to vote	
Author / Product	Weka Team / External Link	

Weka, short for **Waikato Environment for Knowledge Analysis**, is a powerful suite of **machine learning software** developed at the University of Waikato, New Zealand.

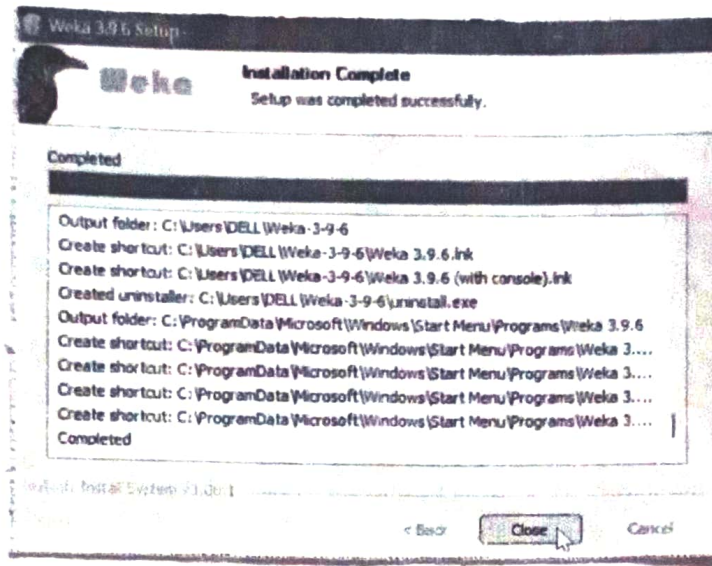
Step 2 It will redirect to a new webpage, click on Start Download. Download will start.



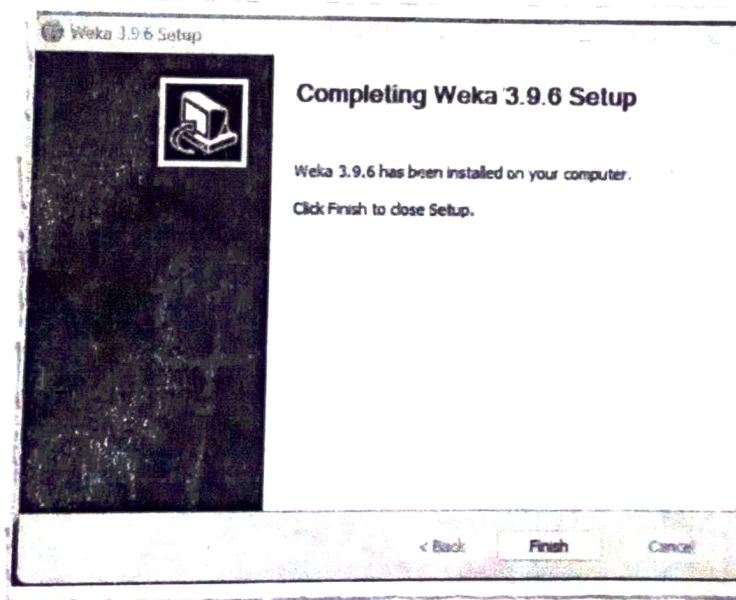
Step 3:- Run the executable file, setup screen will appear. Click on next.



Step 4: Click on close, after the installation process is complete.



Step 5: Click on finish to finish the installation process.



ep 6:- Weka is successfully installed on the system and an icon is created on the desktop.

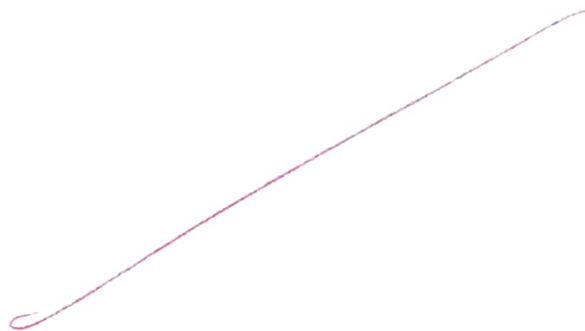


Step 7:- Run the software and see the interface.



LEARNING OUTCOME:

I learned to install WEKA and how it is written in Java.



EXPERIMENT 2

PROBLEM STATEMENT: Perform data pre-processing including cleaning, integration and transformation on ARFF files using WEKA.

THEORY:

Data pre-processing is an essential step in data mining that involves preparing raw data into a clean and suitable format for analysis. Real-world data is often incomplete, noisy and inconsistent, which can affect the accuracy of data mining results. Therefore, data preprocessing improves data quality and enhances the performance of machine learning algorithms.

WEKA is an open-source data mining tool that provides various filters and utilities to perform data pre-processing on datasets in ARFF format. Using WEKA explorer, data can be cleaned, integrated and transformed easily through graphical filters.

Data Cleaning - removes duplicate records, handles missing values, and detect outliers.

Data Integration - creates or combines attributes to improve the quality of information.

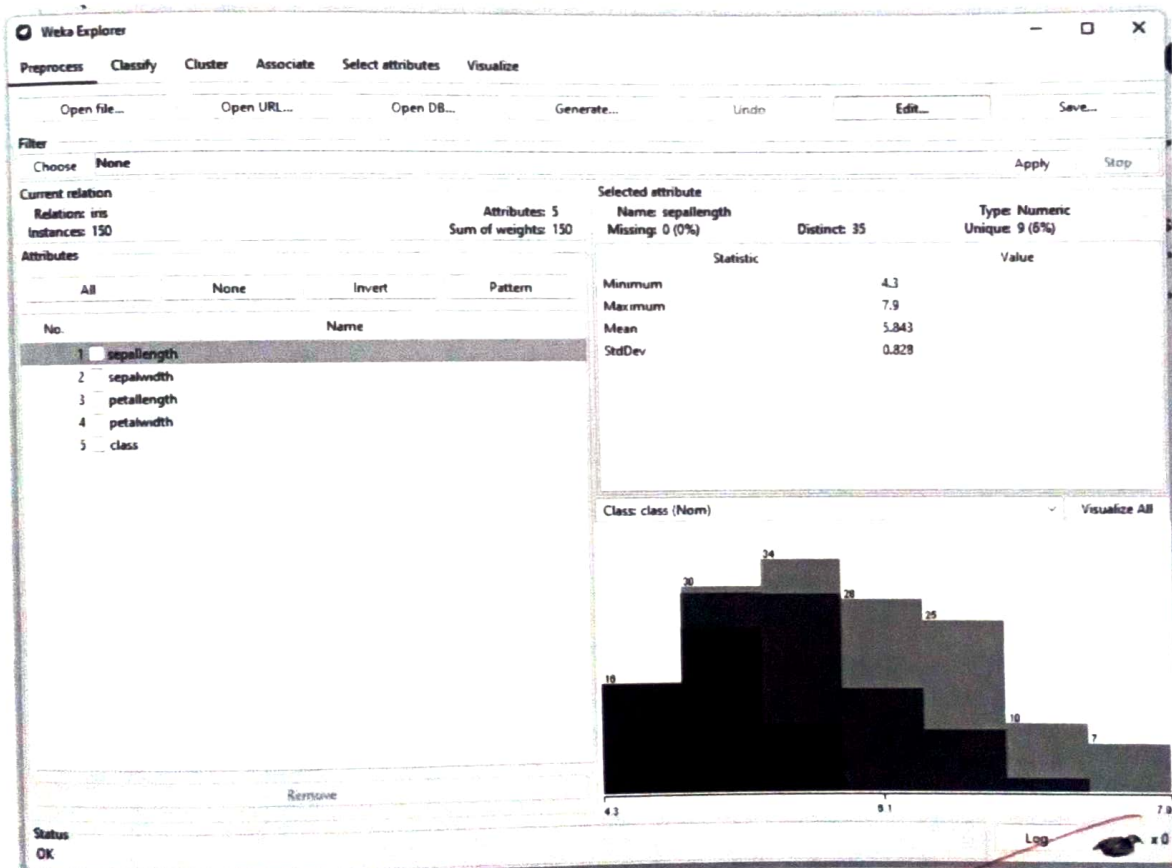
Data transformation - converts data into suitable forms such as normalization, standardization and discretization.

CODE:

Step 1: Open WEKA GUI Chooser.
: Click on Explorer.



Step 2: (i) In WEKA Explorer, go to the Preprocess tab.
(ii) Click open file. Select the iris.arff dataset.
(iii) Dataset details are displayed.
• Instances: 150 • Attributes: 5



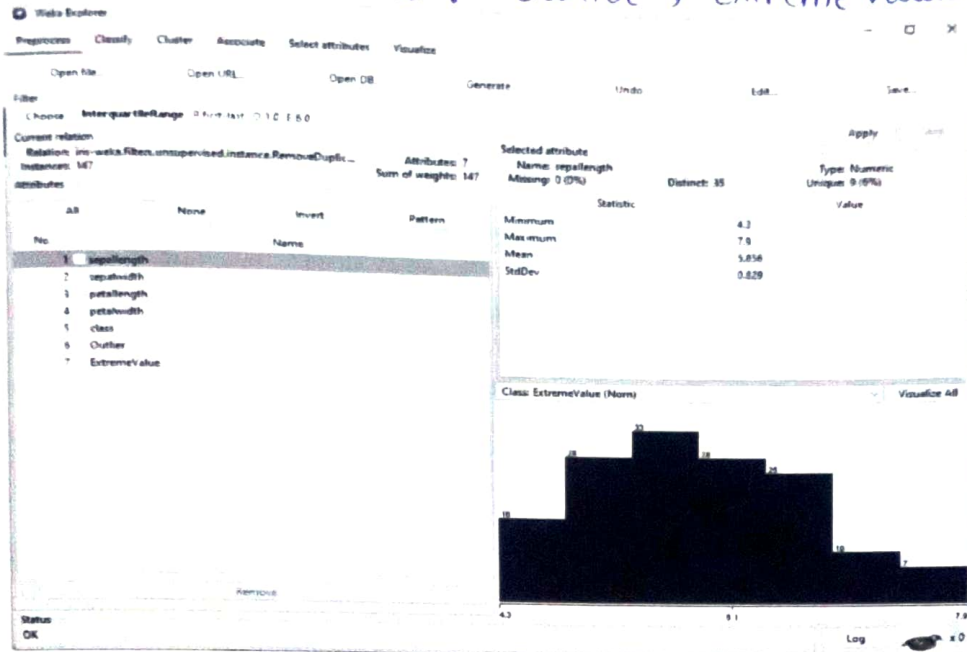
Step 3: (i) Click Choose.

(ii) Select : unsupervised → attribute → Interquartile Range

(iii) keep default parameters.

(iv) Click Apply.

(v) Duplicate records are removed. Two new attributes are added : Outlier , Extreme Value



Step 4: Handle Missing values.

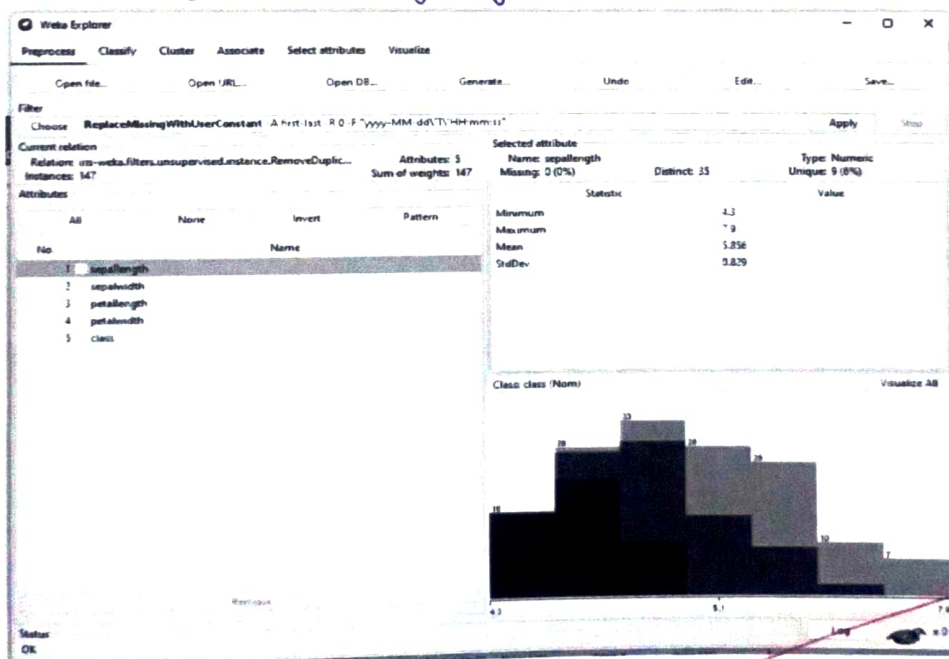
(i) Click Choose.

(ii) Select : unsupervised → attribute → Replace Missing User Constant

(iii) Set default constant value.

(iv) Click Apply.

(v) Missing values (if any are replaced).



Step 5: Remove Duplicate Instances

(i) Under Filter, click Choose

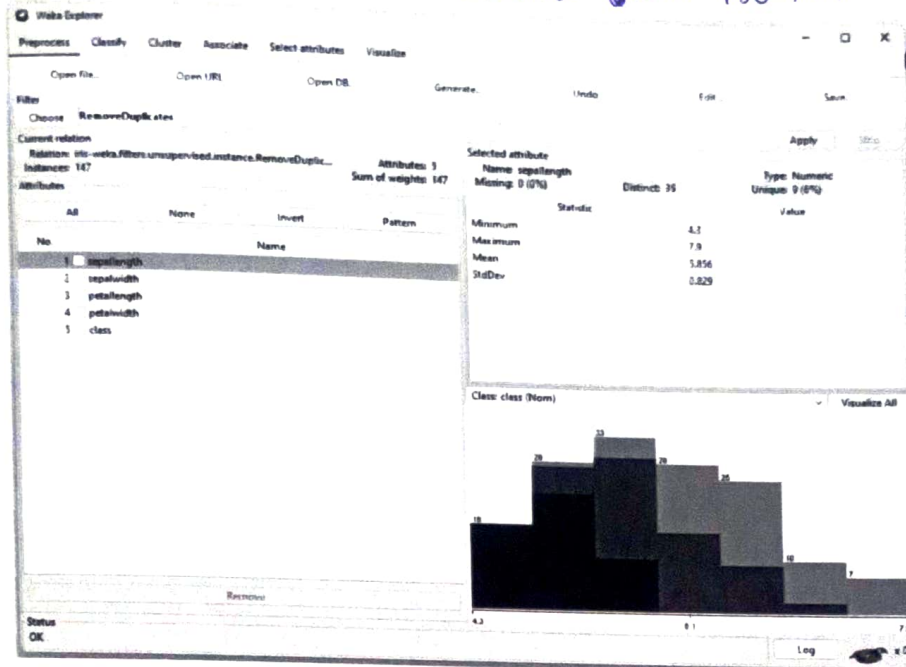
(ii) Select:

unsupervised → instance → Remove Duplicates

(iii) Click Apply.

(iv) Duplicate records are removed.

• Instances reduced from 150-147



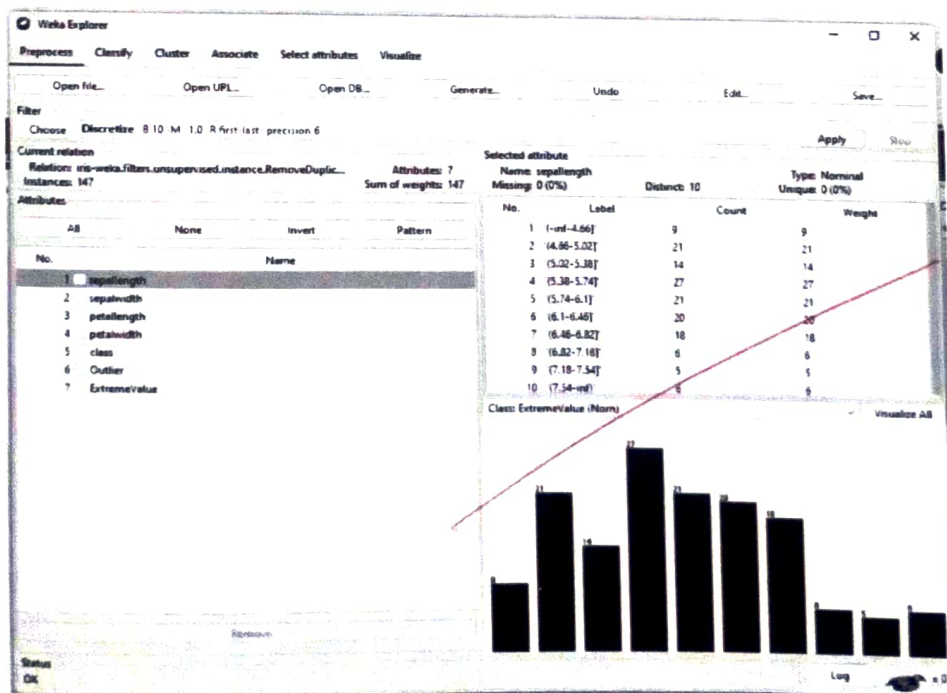
Step 6: Discretization

(i) Click Choose

(ii) Select: unsupervised → attribute → Discretize

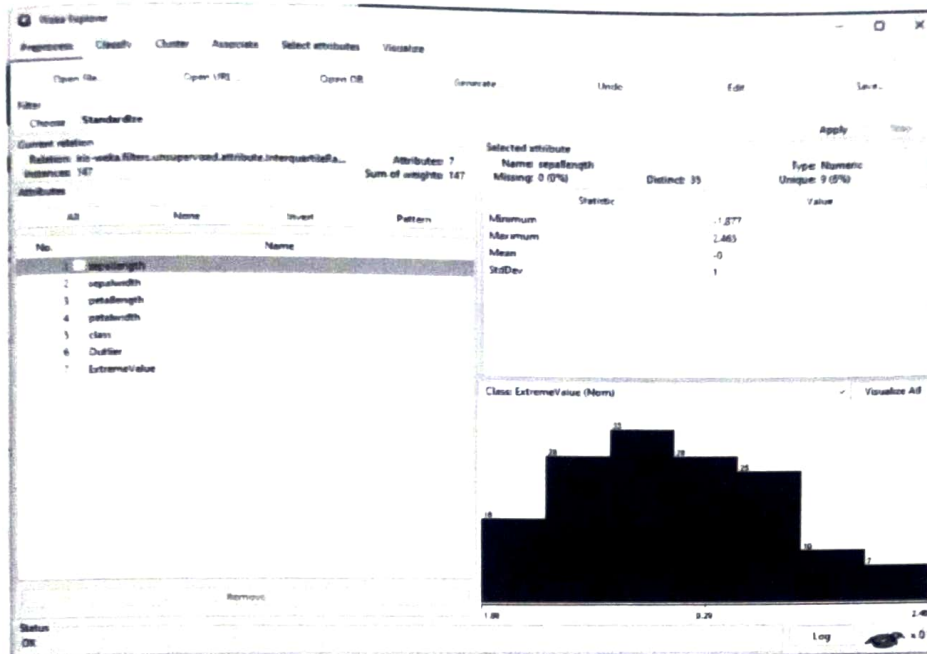
(iii) Set parameters: Bins=10

(iv) click Apply.



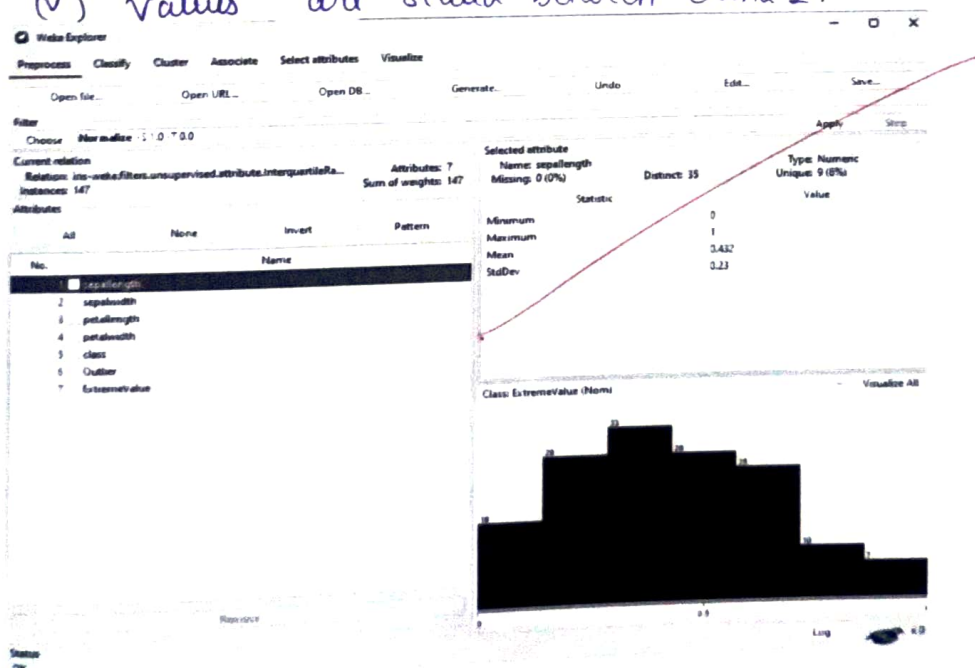
Step 7: Standardization

- Click Choose
- Select: unsupervised → attribute → Standardize
- Click Apply
- Attributes are standardized: Mean ≈ 0 , S.D ≈ 1



Step 8: Normalization

- Click Choose
- Select: unsupervised → attribute → Normalize
- Range set from 0 to 1
- Click Apply
- Values are scaled between 0 and 1.



Step 9 Remove useless Attributes

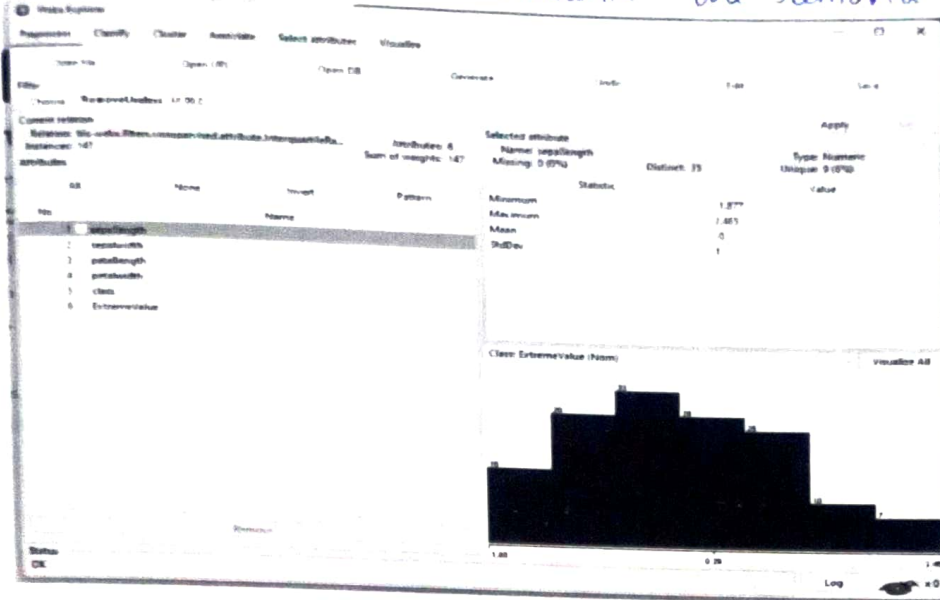
(i) click Choose

(ii) Select

unsupervised → attribute → Remove Useless

(iii) click Apply

(iv) Attributes with low variance are removed



Step 10: Add new Attribute using Expression

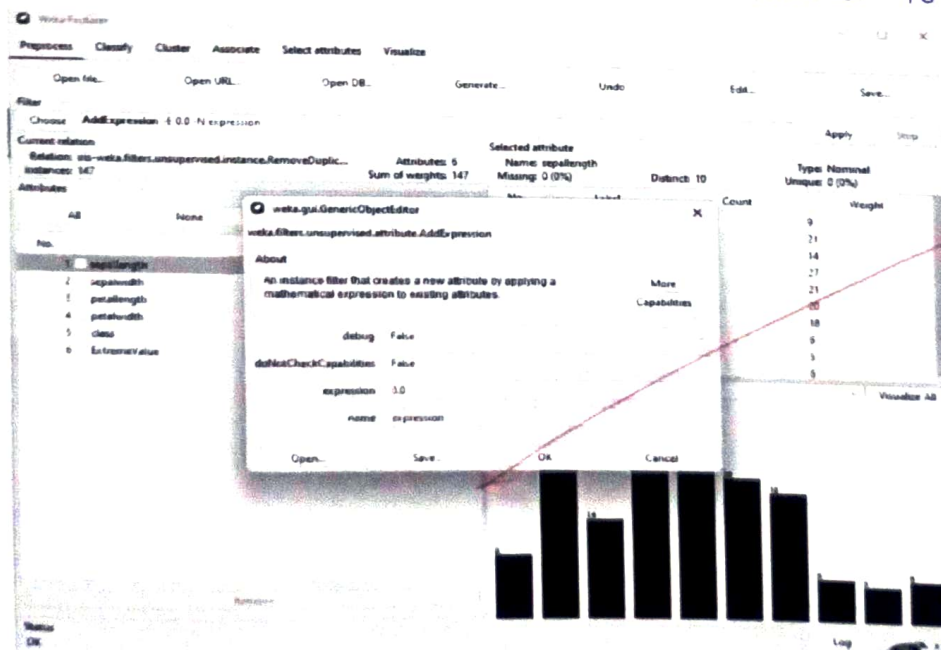
(i) click Choose

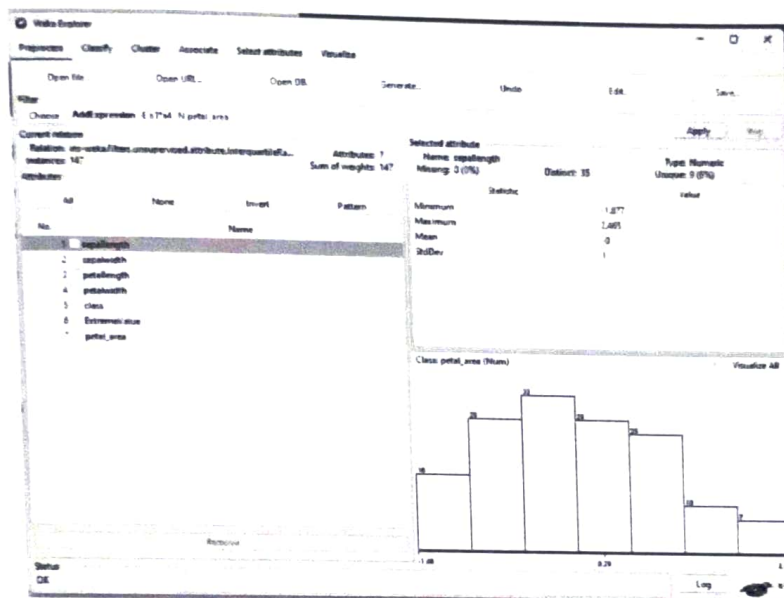
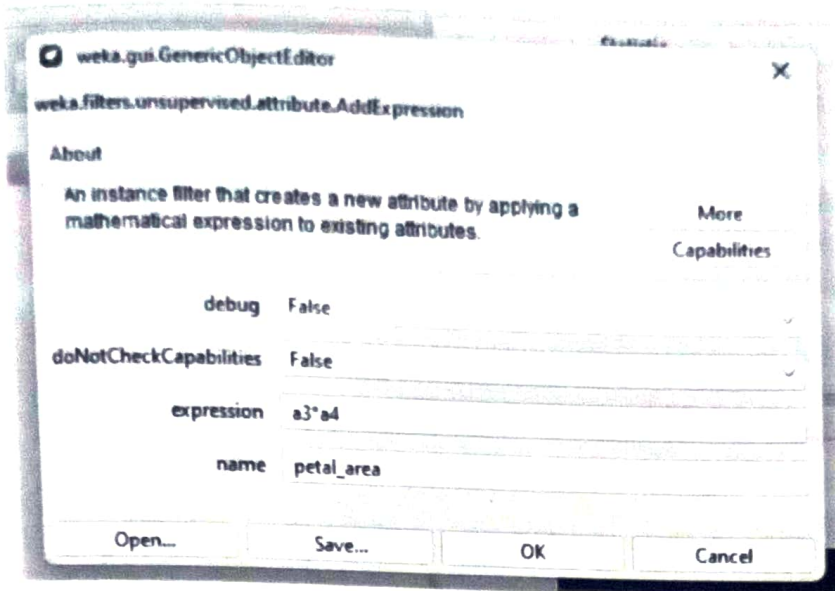
(ii) Select: unsupervised → attribute → Add Expression

(iii) Enter mathematical expression.

(iv) click OK → Apply

(v) A new derived attribute is added to the dataset.





Viewer

Relation: iris-weka.filters.unsupervised.attribute.InterquartileRange-Rfirst-last-Q3.0-E6.0-weka.filters.unsupervised.attribute.ReplaceMissingWithUserConstant-Afirst-last-RD-F-yyyy-MM-d

No.	1: sepal.length	2: sepal.width	3: petal.length	4: petal.width	5: class	6: Extreme value	7: petal_area
	Numeric	Numeric	Numeric	Numeric	Nominal	Nominal	Numeric
1	-0.912390196...	1.016496193...	-1.353110886...	-1.33114906...	iris-se...	no	1.801192287...
2	-1.153615662...	-0.12764577...	-1.353110886...	-1.33114906...	iris-se...	no	1.801192287...
3	-1.394841128...	0.330011015...	-1.409957785...	-1.33114906...	iris-se...	no	1.876863983...
4	-1.515453860...	0.101182622...	-1.296263987...	-1.33114906...	iris-se...	no	1.725520590...
5	-1.033002929...	1.245324586...	-1.353110886...	-1.33114906...	iris-se...	no	1.801192287...
6	-0.550551998...	1.831809764...	-1.182570188...	-1.06725302...	iris-se...	no	1.262101610...
7	-1.515453860...	0.787667800...	-1.353110886...	-1.19920104...	iris-se...	no	1.622651986...
8	-1.033002929...	0.787667800...	-1.296263987...	-1.33114906...	iris-se...	no	1.725520590...
9	-1.736679326...	-0.35647416...	-1.353110886...	-1.33114906...	iris-se...	no	1.801192287...
10	-1.153615662...	0.101182622...	-1.296263987...	-1.46309708...	iris-se...	no	1.896560055...
11	-0.550551998...	1.474152979...	-1.296263987...	-1.33114906...	iris-se...	no	1.725520590...
12	-1.274228395...	0.787667800...	-1.238417087...	-1.33114906...	iris-se...	no	1.649848893...
13	-1.274228395...	-0.12764577...	-1.353110886...	-1.46309708...	iris-se...	no	1.979732587...
14	-1.877282098...	-0.12764577...	-1.523651583...	-1.46309708...	iris-se...	no	2.229250184...
15	-0.08810106...	2.160638157...	-1.466804684...	-1.33114906...	iris-se...	no	1.952535680...
16	-0.188713799...	3.075951728...	-1.296263987...	-1.06725302...	iris-se...	no	1.383441660...
17	-0.550551998...	1.831809764...	-1.409957785...	-1.06725302...	iris-se...	no	1.504781710...
18	-0.912390196...	1.016496193...	-1.253110886...	-1.19920104...	iris-se...	no	1.622651986...
19	-0.188713799...	1.702981371...	-1.182570188...	-1.19920104...	iris-se...	no	1.418139403...
20	-0.912390196...	1.702981371...	-1.296263987...	-1.19920104...	iris-se...	no	1.554481125...
21	-0.550551998...	0.787667800...	-1.182570188...	-1.33114906...	iris-se...	no	1.574377197...
22	-0.912390196...	1.474152979...	-1.296263987...	-1.06725302...	iris-se...	no	1.383441660...
23	1.515453860...	1.245324586...	-1.580490483...	-1.33114906...	iris-se...	no	2.103879073...
24	-0.912390196...	0.558839408...	-1.182570188...	-0.93530500...	iris-se...	no	1.106083816...

Add instance Undo OK Cancel

LEARNING OUTCOME:

I understood that data pre-processing improves the quality and consistency of data, making it suitable for data mining and machine learning tasks.

Don
31/11/26

EXPERIMENT 3

PROBLEM STATEMENT: Apply association rule mining on ARFF files using WEKA.

THEORY:

Association Rule Mining is a data mining technique used to identify relationships and patterns among items in large datasets. It helps in discovering rules of the form $X \rightarrow Y$, indicating that the occurrence of item X implies the occurrence of item Y .

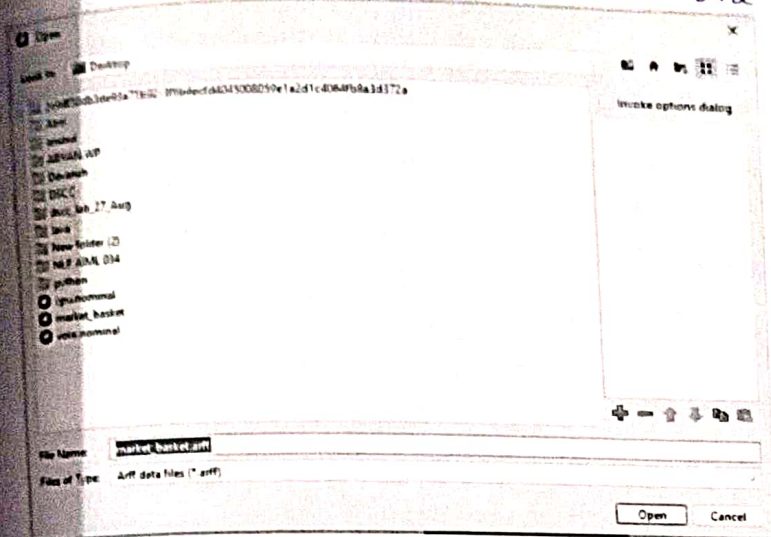
WEKA is an open-source data mining tool that supports various machine learning and data mining algorithms. WEKA uses ARFF (Attribute-Relation File Format) as its standard input format, which contains attribute information and data values.

To apply association rule mining, the ARFF dataset is loaded into WEKA Explorer, the Apriori algorithm is selected from Associate tab and suitable support and confidence values are set. The generated rules help in understanding hidden patterns and associations in the dataset.

CODE:

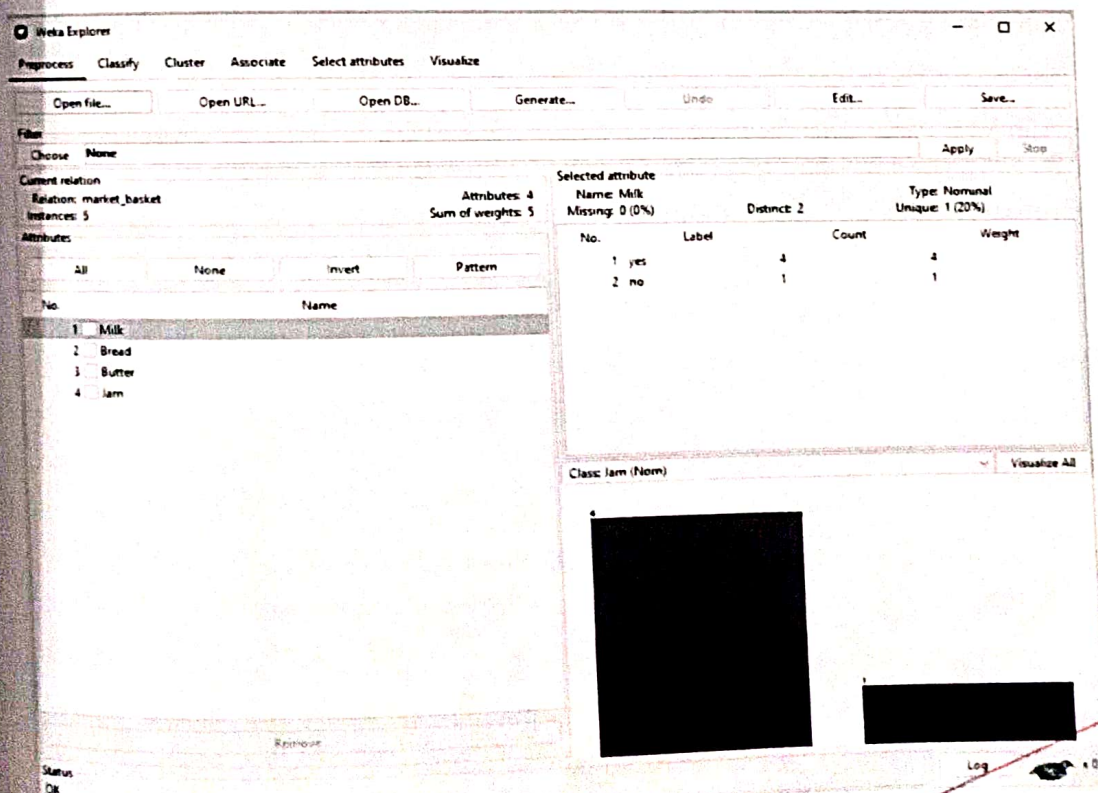
Step 1: Open WEKA Explorer. Click on Open file... in Preprocess tab.

- Select the file market-basket.arff.
- Click open and load the dataset.



Step 2: After loading, the dataset information is displayed:

- Relation name: market-basket
- Number of instances: 5, No. of attributes: 4
- The attributes Milk, Bread, Butter and Jam are shown with nominal value.



- Step 3: Click on the Associate tab.
- select Apriori algorithm.
 - set the parameters as shown:
 - min support = 0.4
 - min confidence = 0.6
 - No. of rules = 10
 - click OK to confirm the settings.

Weka GUI - Generic Object Editor

weka.associations.Apriori

About More

Class implementing an Apriori-type algorithm Capabilities

car False

classIndex -1

delta 0.05

doNotCheckCapabilities False

lowerBoundMinSupport 0.4

metricType Confidence

minMetric 0.6

numRules 10

outputItemSets False

removeAllMissingCols False

significanceLevel -1.0

treatZeroAsMissing False

upperBoundMinSupport 1.0

verbose False

Open Save OK Cancel

- Step 4: Click on Start button.
- WEKA generates frequent itemsets & association rules.
 - Output window is displayed.

Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Associator

Choose Apriori -N 10 -T 0 -C 0.6 -D 0.05 -U 1.0 -M 0.4 -S -1.0 -c -1

Start Stop

Result list (right-click for ...)

13:32:26 - Apriori

Associator output

=== Run information ===

Scheme: weka.associations.Apriori -N 10 -T 0 -C 0.6 -D 0.05 -U 1.0 -M 0.4 -S -1.0 -c -1

Relation: market_basket

Instances: 5

Attributes: 4

Milk

Bread

Butter

Jam

=== Associator model (full training set) ===

Apriori

=====

Minimum support: 0.5 (2 instances)

Minimum metric <confidence>: 0.6

Number of cycles performed: 10

Generated sets of large itemsets:

Size of set of large itemsets L(1): 5

Size of set of large itemsets L(2): 6

Size of set of large itemsets L(3): 4

Best rules found:

1. Jam=yes 3 ==> Butter=no 3 <conf:(1)> lift:(1.25) lev:(0.12) [0] conv:(0.6)

2. Jam=no 2 ==> Milk=yes 2 <conf:(1)> lift:(1.25) lev:(0.08) [0] conv:(0.4)

3. Jam=no 2 ==> Bread=yes 2 <conf:(1)> lift:(1.25) lev:(0.08) [0] conv:(0.4)

4. Bread=yes Jam=no 2 ==> Milk=yes 2 <conf:(1)> lift:(1.25) lev:(0.08) [0] conv:(0.4)

5. Milk=yes Jam=no 2 ==> Bread=yes 2 <conf:(1)> lift:(1.25) lev:(0.08) [0] conv:(0.4)

6. Jam=no 2 ==> Milk=yes Bread=yes 2 <conf:(1)> lift:(1.25) lev:(0.16) [0] conv:(0.8)

7. Milk=yes Jam=yes 2 ==> Butter=no 2 <conf:(1)> lift:(1.25) lev:(0.08) [0] conv:(0.4)

8. Bread=yes Jam=yes 2 ==> Butter=no 2 <conf:(1)> lift:(1.25) lev:(0.08) [0] conv:(0.4)

9. Bread=yes 4 ==> Milk=yes 3 <conf:(0.75)> lift:(0.94) lev:(-0.04) [0] conv:(0.4)

10. Bread=yes 4 ==> Milk=yes 3 <conf:(0.75)> lift:(0.94) lev:(-0.04) [0] conv:(0.4)

LEARNING OUTCOME:

I understood association rules, successfully generated them for the market basket dataset using the Apriori algorithm in WEKA.

Q

EXPERIMENT 4

PROBLEM STATEMENT: Implementation of Neural Network technique on ARFF files using WEKA.

THEORY:

A Neural Network is a machine learning technique inspired by the human brain, consisting of interconnected neurons arranged in input, hidden and output layers. It is used for classification and prediction tasks by learning patterns from data.

In this experiment, a Multilayer Perceptron (MLP) neural network is implemented using WEKA. WEKA is an open-source tools that provides various machine learning algorithms through an easy-to-use graphical interface.

The dataset used is in ARFF, which defines attributes and data instances in a structured manner.

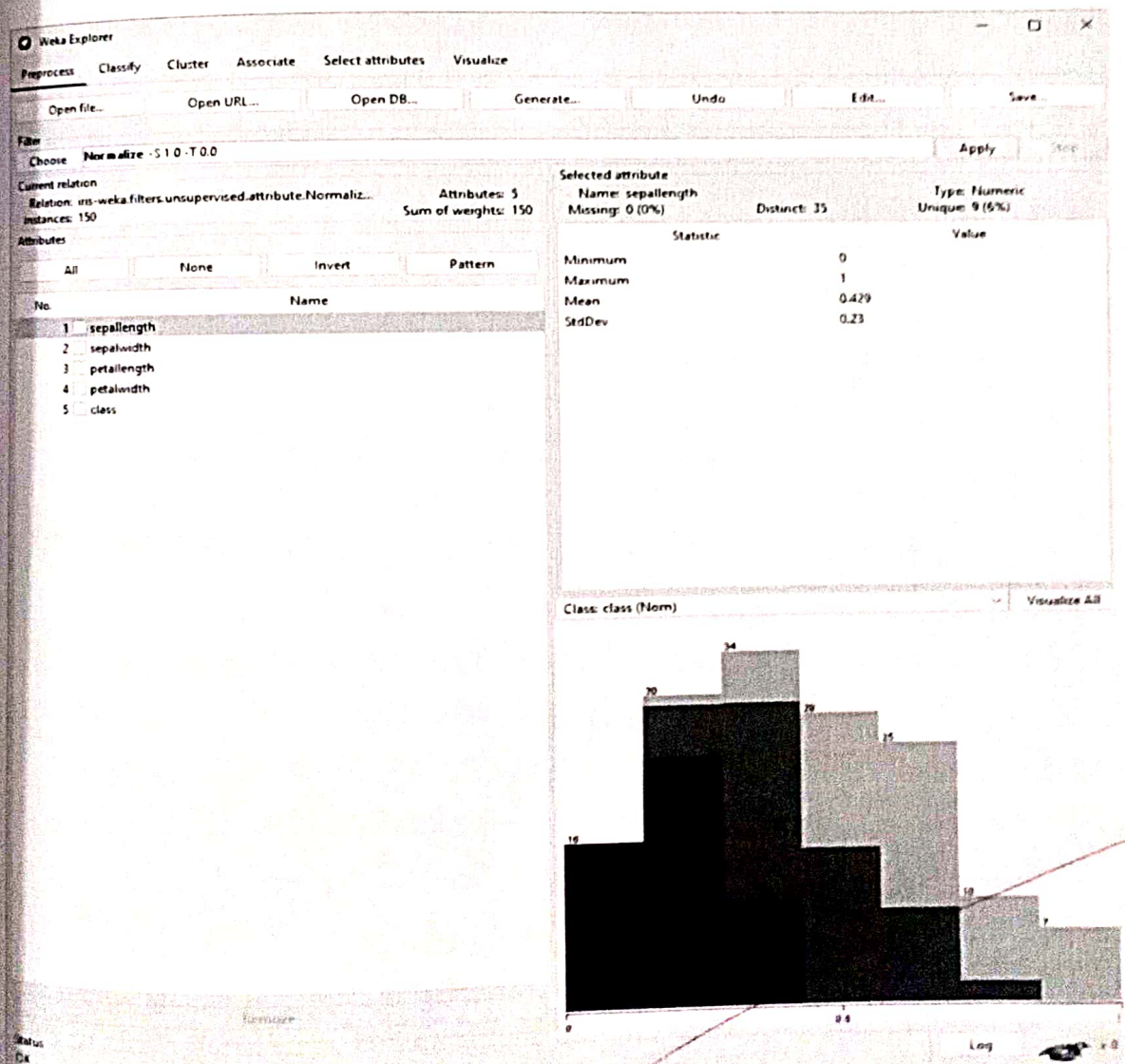
The dataset used in ARFF uses MLP. The MLP in WEKA uses the backpropagation algorithm to train the network by minimizing error through iterative weight adjustment.

CODE:

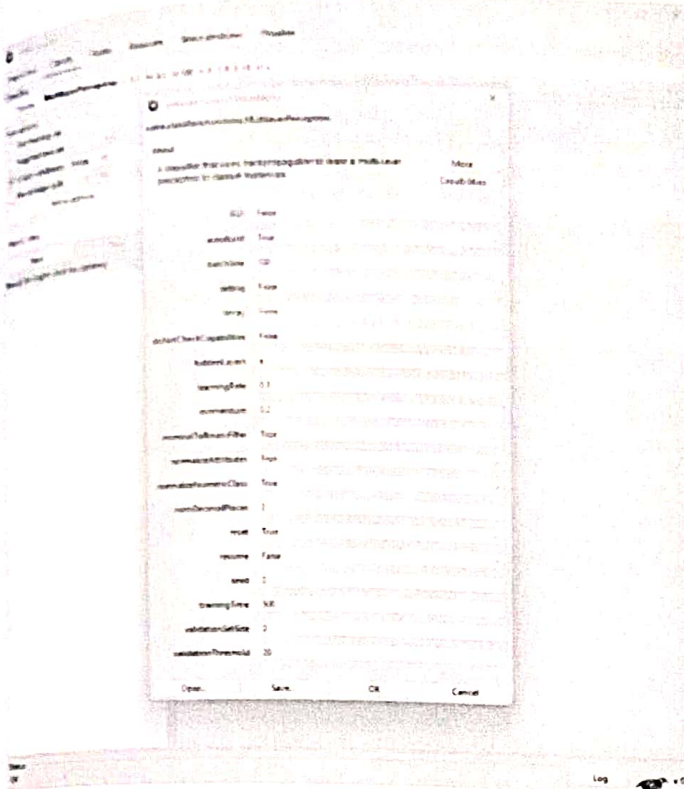
Step 1:- Open WEKA. Launch application. Click Explorer.
Go to preprocess tab. Open .arff dataset.



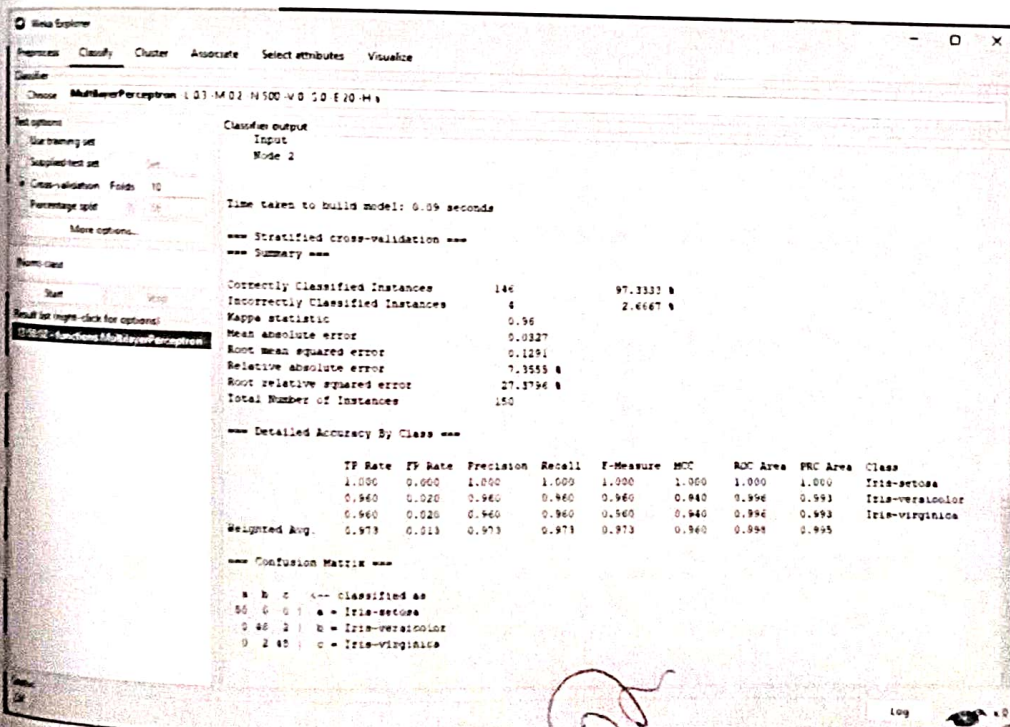
Step 2:- Select filter → Choose. Apply Normalize filter.
Click Apply.



Step 3- Move to Classify tab.
 Click Choose under Classifier.
 Select → function → Multilayer Perceptron (Neural Network)



Step 4. Set Neural Network Parameters.
 Select Cross-validation. Set no. of folds.
 Click Start.



LEARNING OUTCOME:

I learned to implement and evaluate a Neural Network on an ARFF dataset using WEKA.

EXPERIMENT 5

PROBLEM STATEMENT: Implementation of Bagging and Boosting techniques on ARFF files using WEKA.

THEORY:

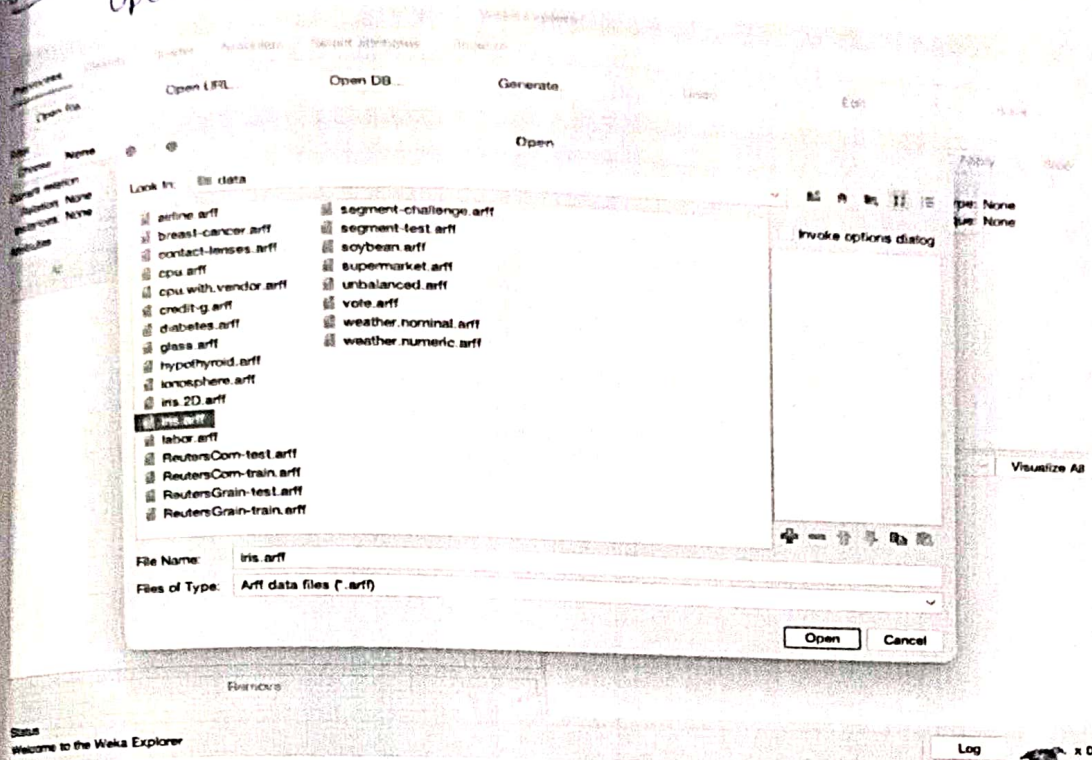
Ensemble learning techniques combine multiple classifiers to improve prediction accuracy. In this experiment, Bagging & Boosting methods are implemented using datasets stored in ARFF.

Bagging creates multiple training samples from the original dataset using random sampling with replacement. Separate models are trained on each sample and the final result is obtained through majority voting.

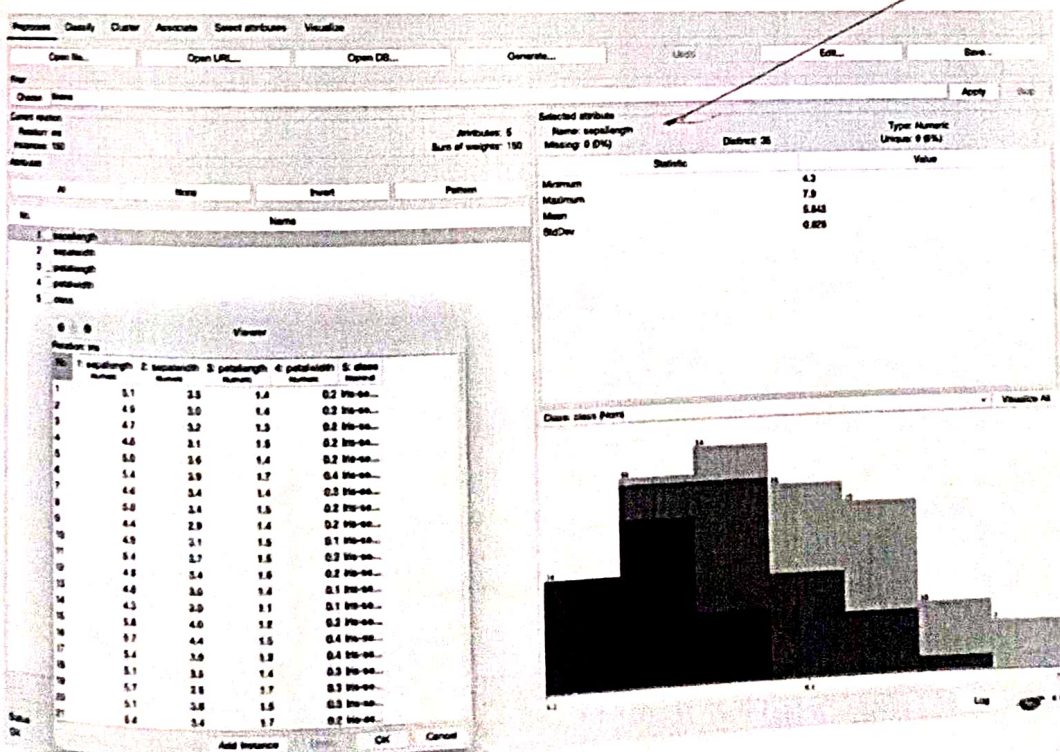
Boosting builds classifiers sequentially, where each new model focuses on correcting the errors of the previous one. Misclassified instances are given higher importance, improving overall accuracy. AdaBoost is commonly used Boosting algorithm.

ARFF files store dataset information including attributes and data instances, which can be easily loaded into WEKA for classification & evaluation.

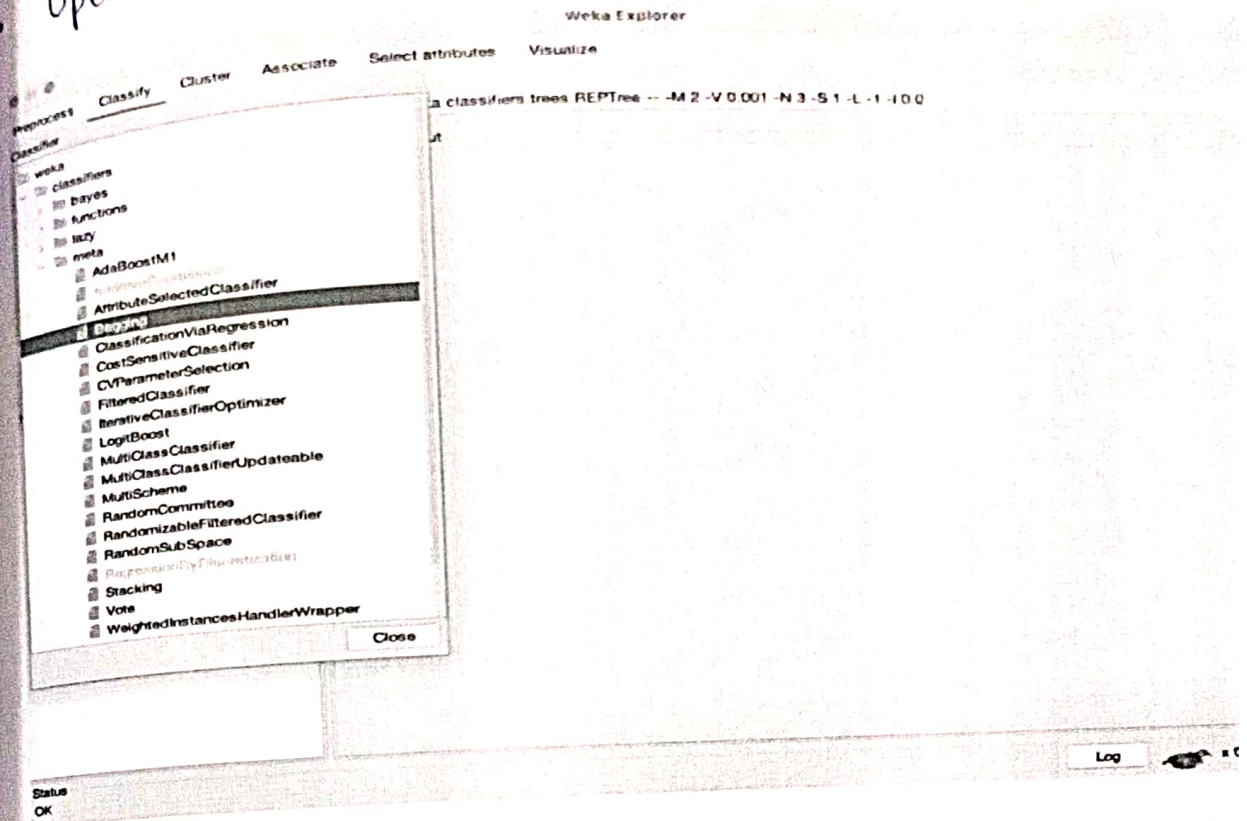
CODE:
 Step 1- open weka → Preprocess tab.
 open iris.arff.



Step 2:-
 select attributes in Preprocess.
 view statistics and histogram.
 check dataset instances.

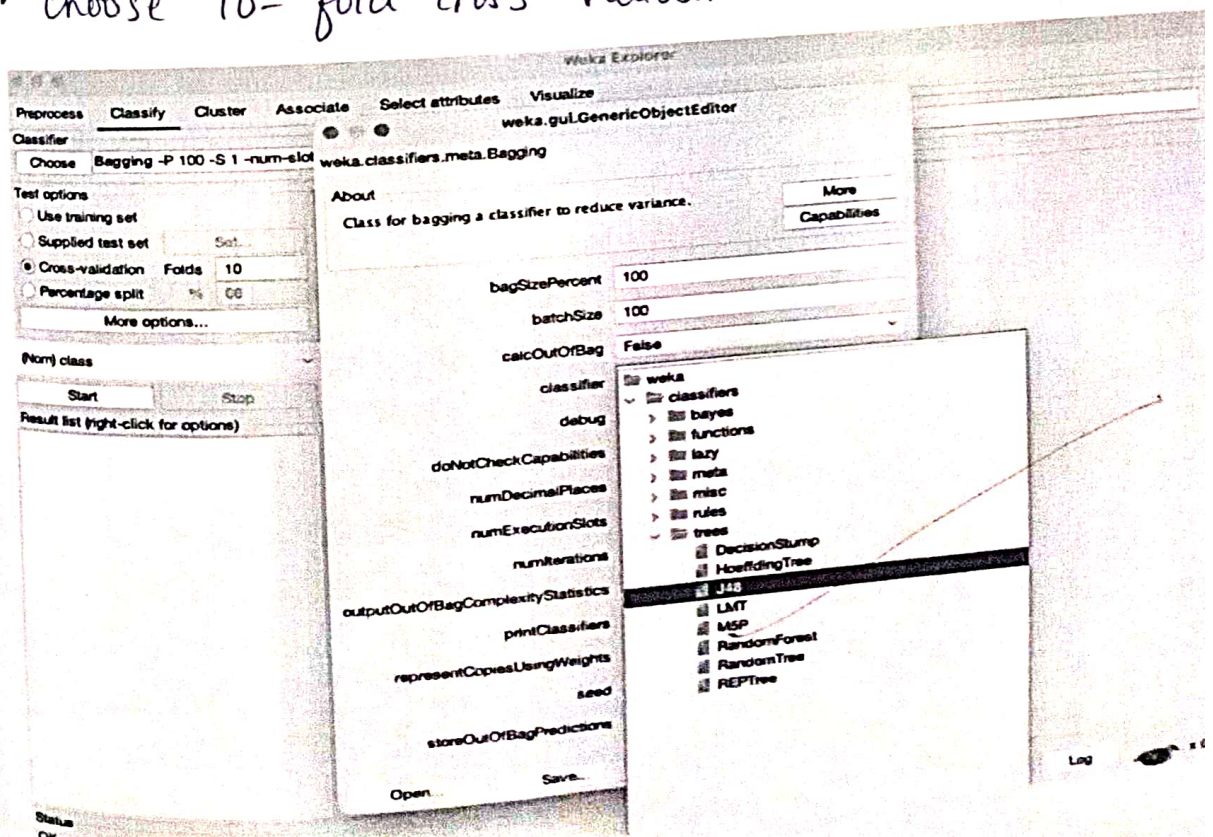


- Step 3- Select bagging.
- Go to classify tab.
 - Choose meta → Bagging.
 - Open classifier settings.



Step 4- Configure Bagging

- Select base classifier. (J48)
- Set parameters
- Choose 10-fold cross validation. Click OK.



Step 5 - Run Bagging
 click start
 Model is trained

Preprocess Classify Cluster Associate Select attributes Visualize

Classifier: Bagging P 100 S 1 num-slots 1 T 10 W weka.classifiers.trees.J48 -C 0.25 -M 2

Classifier output

Run information

Scheme: weka.classifiers.meta.Bagging P 100 -S 1 -num-slots 1 -T 10 -W weka.classifiers.trees.J48 -C 0.25 -M 2

Instances: 150

Attributes: 5
 sepallength
 sepalwidth
 petallength
 petalwidth
 class

Test mode: 10-fold cross-validation

Classifier model (full training set)

Bagging with 10 iterations and base learner

weka.classifiers.trees.J48 -C 0.25 -M 2

Time taken to build model: 0.02 seconds

Stratified cross-validation

Summary

Correctly Classified Instances	142	94.6667 %
Incorrectly Classified Instances	8	5.3333 %
Kappa statistic	0.92	
Mean absolute error	0.0449	
Root mean squared error	0.1673	
Relative absolute error	10.0924 %	
Root relative squared error	35.4791 %	
Total Number of Instances	150	

Detailed Accuracy By Class

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
Weighted Avg.	0.947	0.027	0.948	0.947	0.947	0.920	0.908	0.975	
	0.960	0.040	0.920	0.920	0.940	0.890	0.903	0.905	Iris-setosa
	0.920	0.040	0.923	0.960	0.941	0.911	0.900	0.875	Iris-versicolor
	0.960	0.040	0.923	0.960	0.941	0.911	0.900	0.875	Iris-virginica

Confusion Matrix

	a	b	c	← classified as
a	48	2	0	a = Iris-setosa
b	0	46	4	b = Iris-versicolor
c	0	2	48	c = Iris-virginica

Step 6 - Select AdaBoost

- Click Choose.
- Select meta → AdaBoostM1
- Open settings
- Select base classifier.

Preprocess Classify Cluster Associate Select attributes Visualize

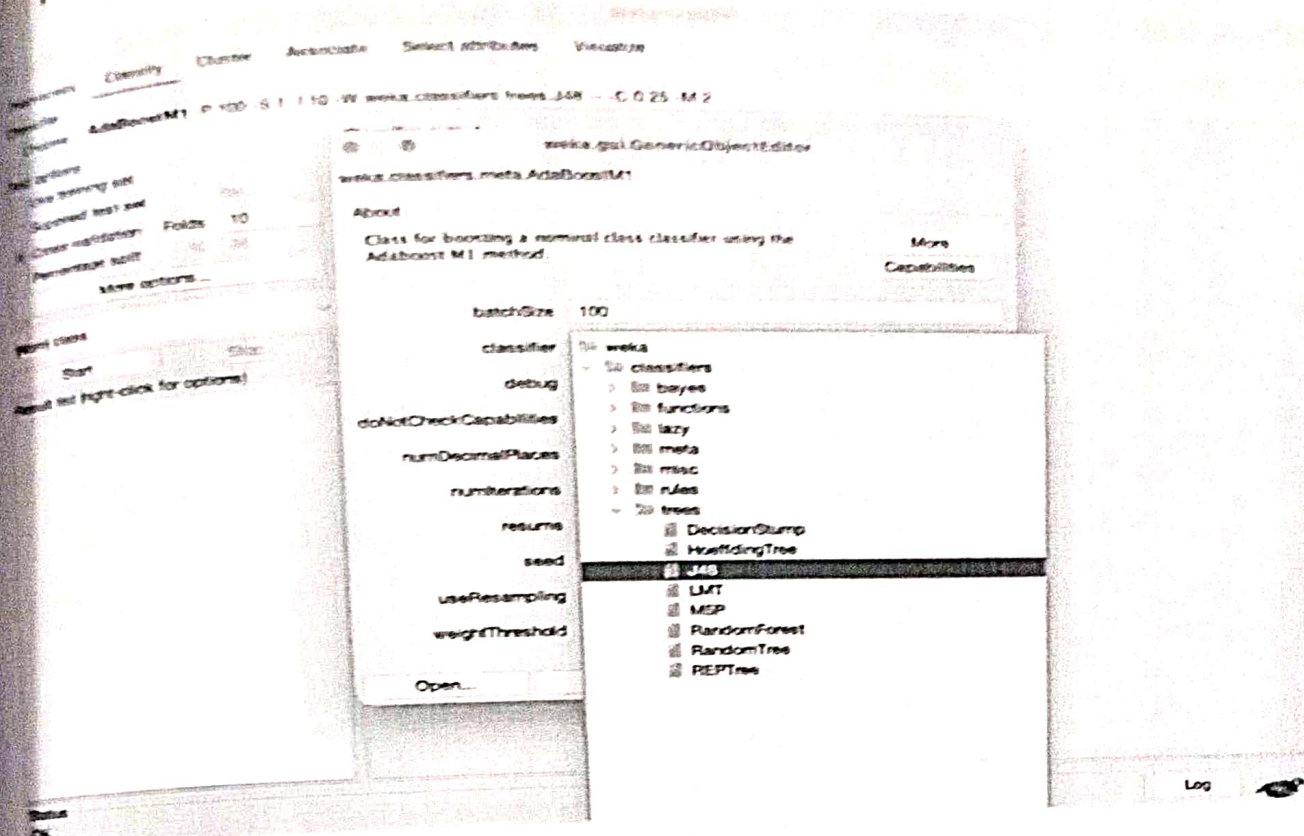
Classifier

weka

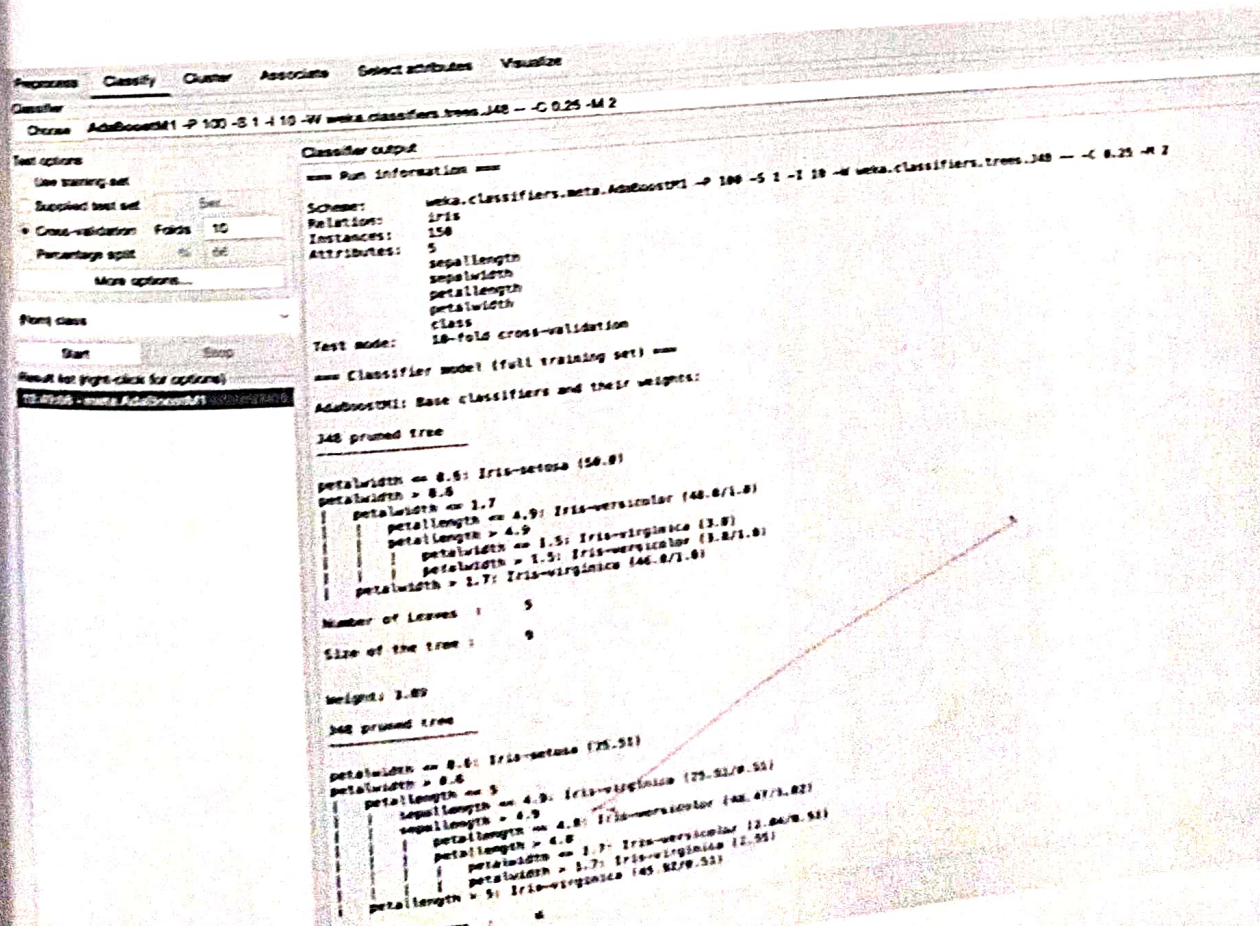
- classifiers
 - bayes
 - functions
 - lazy
 - meta
 - AdaBoostM1
 - AdditiveRegression
 - AttributeSelectedClassifier
 - Bagging
 - ClassificationViaRegression
 - CostSensitiveClassifier
 - CVPParameterSelection
 - FilteredClassifier
 - IterativeClassifierOptimizer
 - LogitBoost
 - MultiClassClassifier
 - MultiClassClassifierUpdateable
 - MultiScheme
 - RandomCommittee
 - RandomizableFilteredClassifier
 - RandomSubSpace
 - RegressionCostFunction
 - Stacking
 - Vote
 - WeightedInstancesHandlerWrapper

Close

Step 7 - Run AdaBoost
 Select cross-validation
 click Start



Step 8 - View Results.
 check accuracy and errors.




```

Preprocess Classify Cluster Associate Select attributes Visualize
Classifier
Classifier AdaBoostM1 P 100 S 1 1 10 W weka classifiers trees J48 -- -C 0.25 -M 2
Options
More options...
Use training set
Supplied test set
Cross validation Folds 10
Percentage split 95 50
More options...
Load class
Start
Result list (right-click for options)
13.04.08 - weka.AdaBoostM1

Classifier output
petalwidth > 1
| petalwidth <= 1.6
| | petalwidth <= 1.5: Iris-virginica (13.78/1.89)
| | petalwidth > 1.5: Iris-versicolor (3.76)
| | petalwidth > 1.6
| | | sepalwidth <= 3.1: Iris-virginica (48.64)
| | | sepalwidth > 3.1
| | | | petallength <= 4.9: Iris-versicolor (2.94)
| | | | petallength > 4.9: Iris-virginica (18.9)
| sepalwidth > 6.5
| | petallength <= 5: Iris-versicolor (43.26)
| | petallength > 5: Iris-virginica (15.58)

Number of Leaves : 8
Size of the tree : 15

Weight: 4.29
Number of performed iterations: 10

Time taken to build model: 0.03 seconds

=== Stratified cross-validation ===
=== Summary ===

Correctly Classified Instances 140 93.333 %
Incorrectly Classified Instances 10 6.667 %
Kappa statistic 0.9
Mean absolute error 0.0467
Root mean squared error 0.2133
Relative absolute error 10.4978 %
Root relative squared error 45.2448 %
Total Number of Instances 150

=== Detailed Accuracy By Class ===

TP Rate FP Rate Precision Recall F-Measure MCC ROC Area PRC Area Class
0.980 0.000 1.000 0.950 0.990 0.985 0.989 0.993 Iris-setosa
0.920 0.060 0.885 0.920 0.902 0.852 0.948 0.875 Iris-versicolor
0.900 0.040 0.910 0.900 0.909 0.864 0.945 0.903 Iris-virginica
Weighted Avg. 0.933 0.033 0.934 0.933 0.934 0.900 0.958 0.973

=== Confusion Matrix ===
a b c ← classified as
49 1 0 | a = Iris-setosa
0 46 4 | b = Iris-versicolor
0 5 45 | c = Iris-virginica

```

LEARNING OUTCOME:
Implemented and compared Bagging & Boosting classifier using weka.

EXPERIMENT 6

PROBLEM STATEMENT: Apply the concept of Voting ensemble method to ARFF files and compare the results with single classifiers.

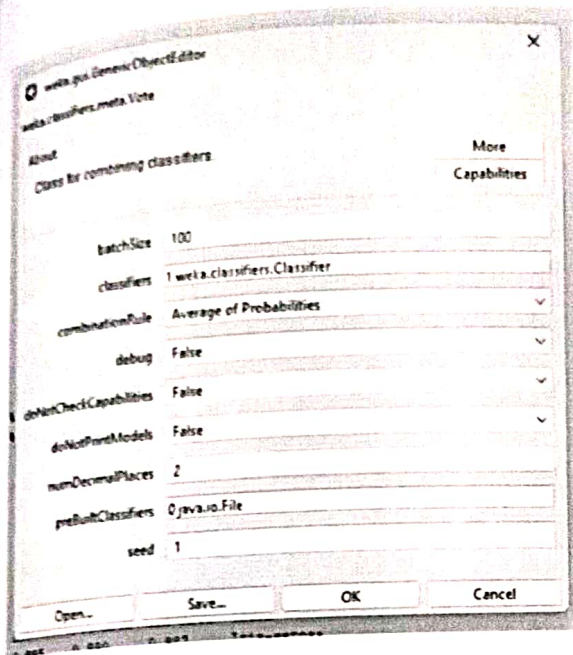
THEORY:

Ensemble learning is a machine learning technique in which classifiers are combined to improve prediction performance. The Voting Ensemble Method is a simple ensemble approach where several classifiers are trained on the same dataset and their predictions are combined to produce a final result.

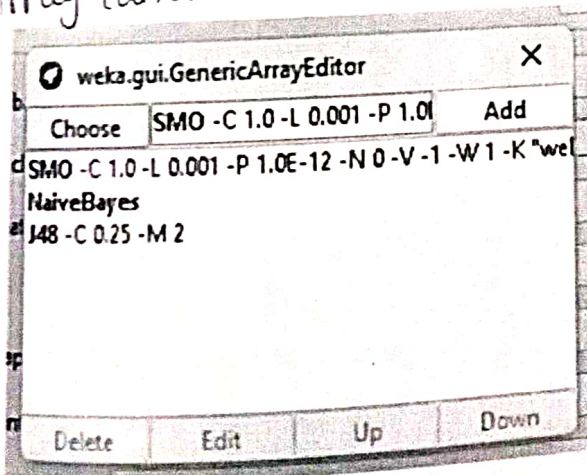
In voting, each classifier gives its prediction and the final class is selected based on majority voting or average probability. This method reduces errors because different classifiers learn different patterns from the data.

Thus, voting ensemble methods generally provide more reliable results compared to single classifiers.

Step 3: open the vote classifier setting window and set the combination rule as Average of Probabilities.



Step 4: Click on the classifier settings with option inside vote and add multiple base classifiers such as SMO, Naive Bayes and J48 using Generic Array Editor.



Step 5:-
 Click OK and run the voting ensemble model. Observe and record the performance results and compare them with single classifier output.

Weka Explorer

Classify Cluster Associate Select attributes Visualize

Classifier: Vote - S1 - B "weka.classifiers.functions.SMO - C 1.0 - L 0.001 - P 1.0E-12 - N 0 - V - 1 - W 1 - K Y" weka.classifiers.functions.supportVector.PolyKernel - E 1.0 - C 250000% - calibrator "weka.classif

Test options:
 Use training set: ☒
 Supplied test set: ☐
 Cross-validation: Folds: 10
 Percentage split: 66
 More options...

(Normal) class:
 Start Stop
 Result list (right-click for options):
 1240220 - trees.J48
 1042136 - meta Vote

Classifier output

J48 pruned tree

```

petalwidth <= 0.6: Iris-setosa (50.0)
petalwidth > 0.6
| petalwidth <= 1.7
| | petalwidth <= 4.9: Iris-versicolor (48.0/1.0)
| | petalwidth > 4.9
| | | petalwidth <= 1.5: Iris-virginica (3.0)
| | | petalwidth > 1.5: Iris-versicolor (3.0/1.6)
| | petalwidth > 1.7: Iris-virginica (46.0/1.0)

```

Number of Leaves : 5
 Size of the tree : 5

Time taken to build model: 0.03 seconds

--- Stratified cross-validation ---
 --- Summary ---

Correctly Classified Instances	145	96.6667 %
Incorrectly Classified Instances	5	3.3333 %
Kappa statistic	0.95	
Mean absolute error	0.1001	
Root mean squared error	0.1623	
Relative absolute error	22.5234 %	
Root relative squared error	34.4359 %	
Total Number of Instances	150	

--- Detailed Accuracy By Class ---

	TP Rate	FP Rate	Precision	Recall	F-Measure	ROC Area	PRC Area	Class
	1.000	0.000	1.000	1.000	1.000	1.000	1.000	Iris-setosa
	0.940	0.020	0.959	0.940	0.949	0.925	0.955	Iris-versicolor
	0.960	0.030	0.961	0.960	0.950	0.925	0.962	Iris-virginica
Weighted Avg.	0.967	0.017	0.967	0.967	0.967	0.950	0.979	

--- Confusion Matrix ---

```

a b c <-- classified as
50 0 0 | a = Iris-setosa
0 47 3 | b = Iris-versicolor
0 2 48 | c = Iris-virginica

```

Status: OK

LEARNING OUTCOME:

Through this experiment I learnt how to apply the voting ensemble method on ARFF datasets and compare its performance with individual classifiers. to improve classification accuracy.

EXPERIMENT 7

PROBLEM STATEMENT: Implementation of Visualization technique on ARFF files using WEKA.

THEORY:

Data Mining is the process of extracting meaningful patterns and knowledge from large datasets. Visualization techniques are essential in data mining as they help represent data graphically, making it easier to understand complex relationships, trends and distributions.

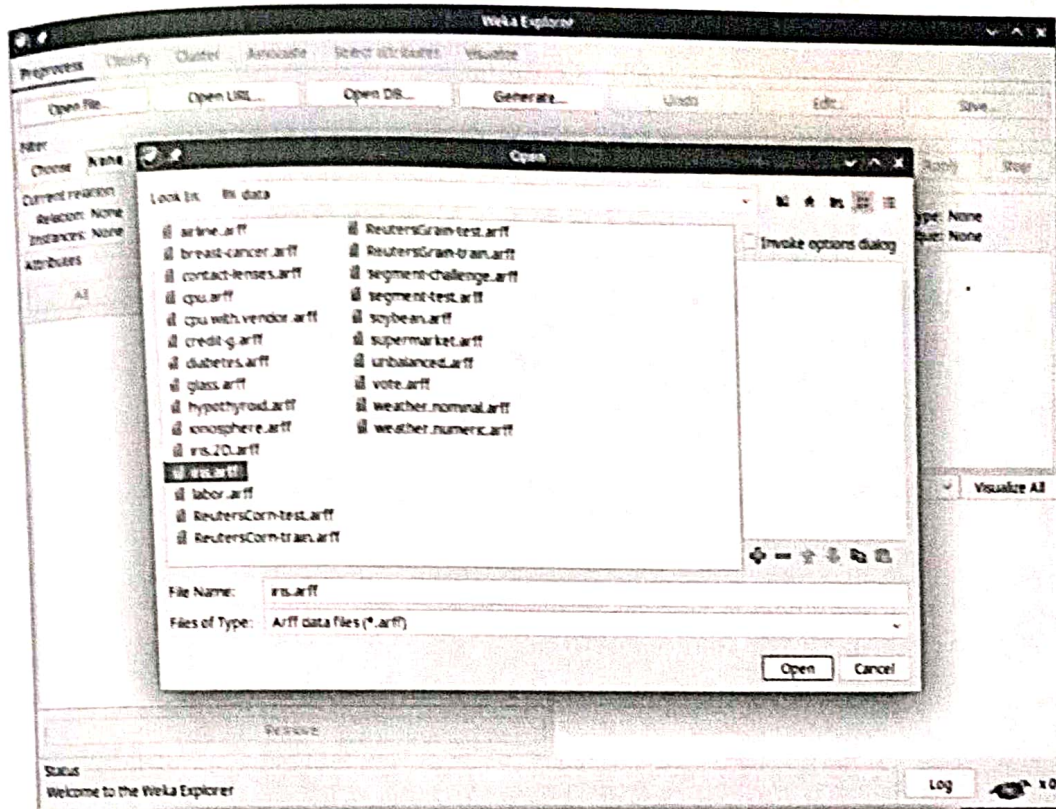
In this experiment we performed on ARFF files using Weka, a widely used data mining tool. ARFF files contain both the dataset and its metadata, including attribute names, types and data instances, which makes them suitable for analysis in Weka.

Weka provides built-in visualization tools such as scatter plots, histograms and attribute-class relationship graphs. These tools allow users to explore data visually, identify correlations between attributes, detect outliers and analyze class distribution.

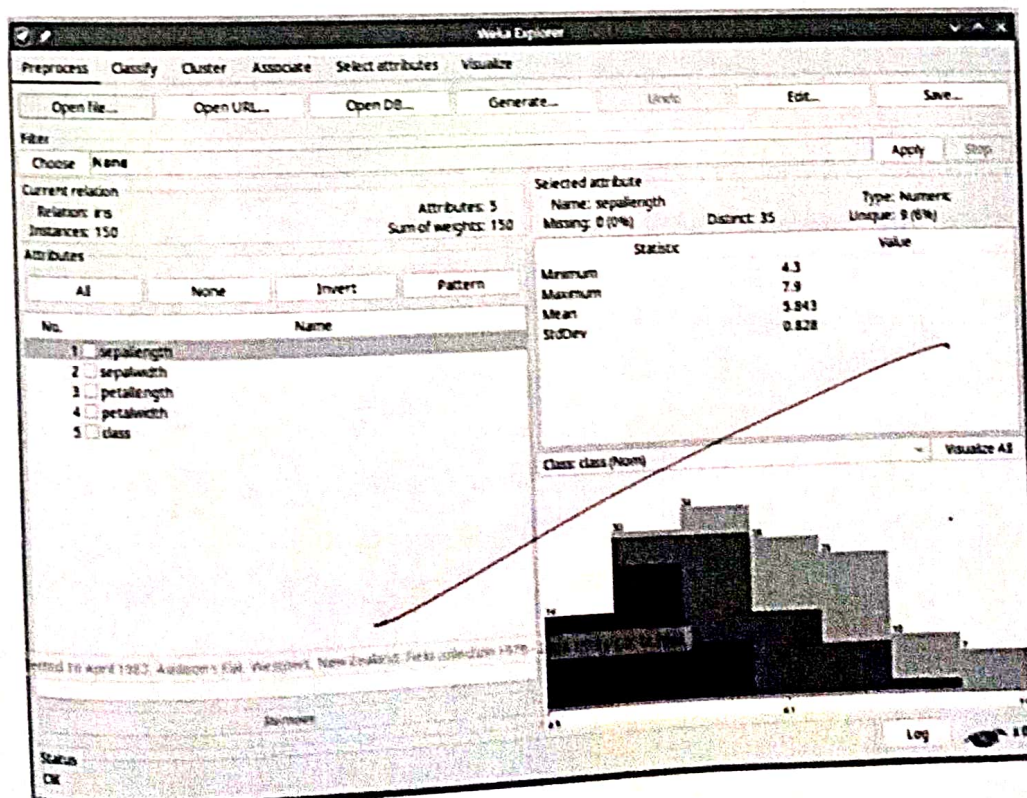
Visualization helps in better data preprocessing, feature selection and understanding of datasets before applying data mining algorithms.

CODE:

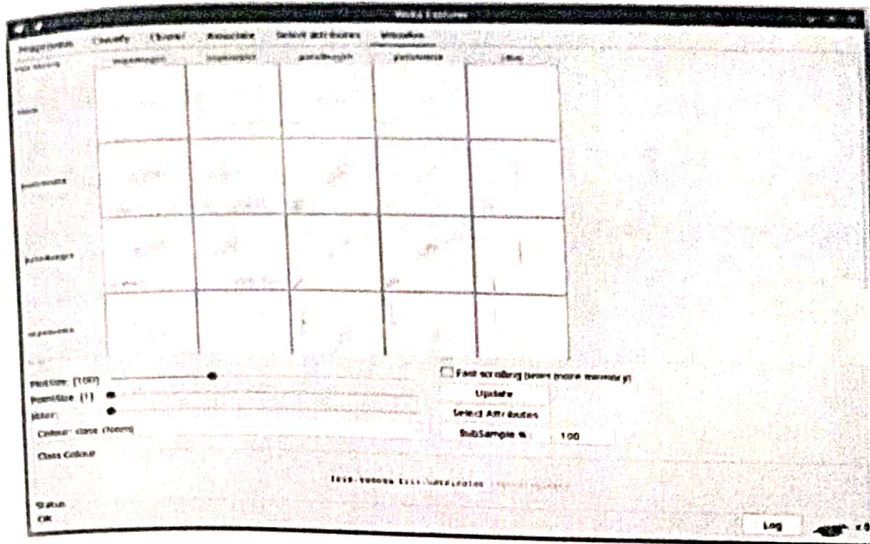
open WEKA and select the Explorer option.
click on open file to load the ARFF dataset.



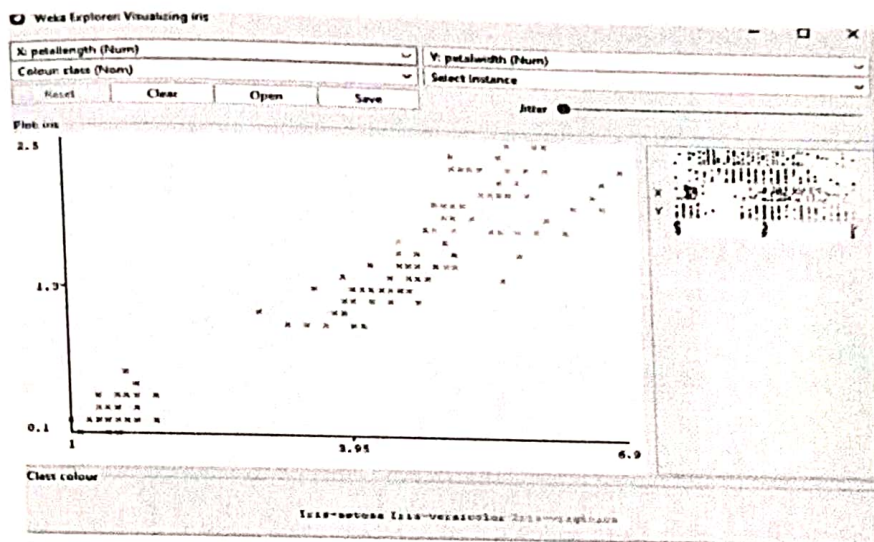
View the dataset details in the Preprocess tab



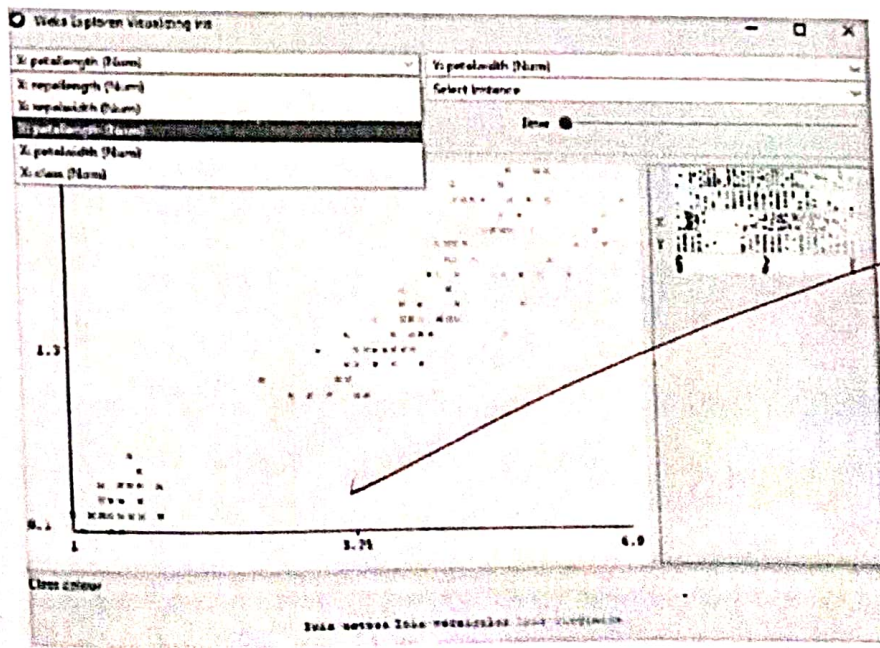
3. click of visualize All to view scatter plot matrix



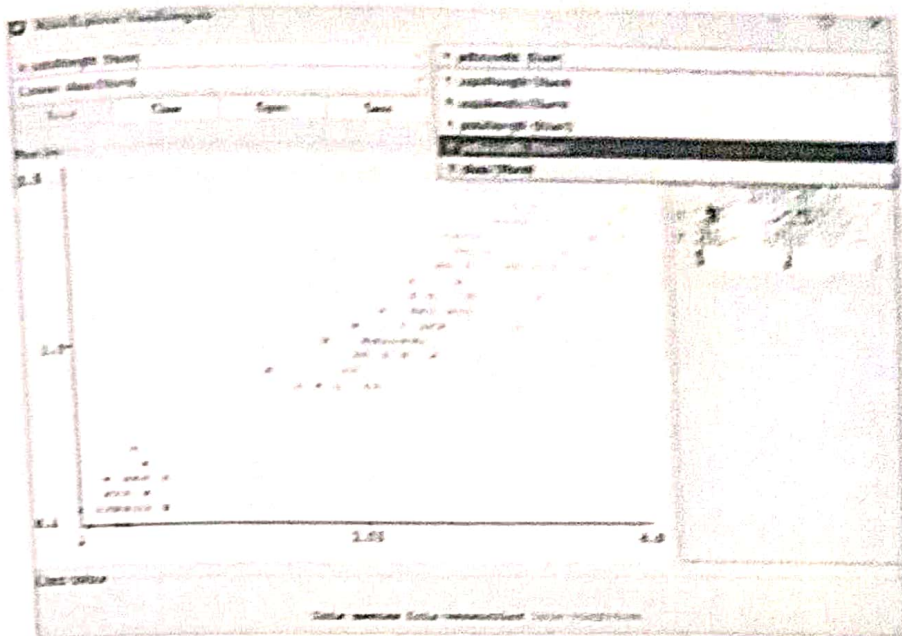
4. Select specific attributes (X and Y axis) to visualize data distribution.



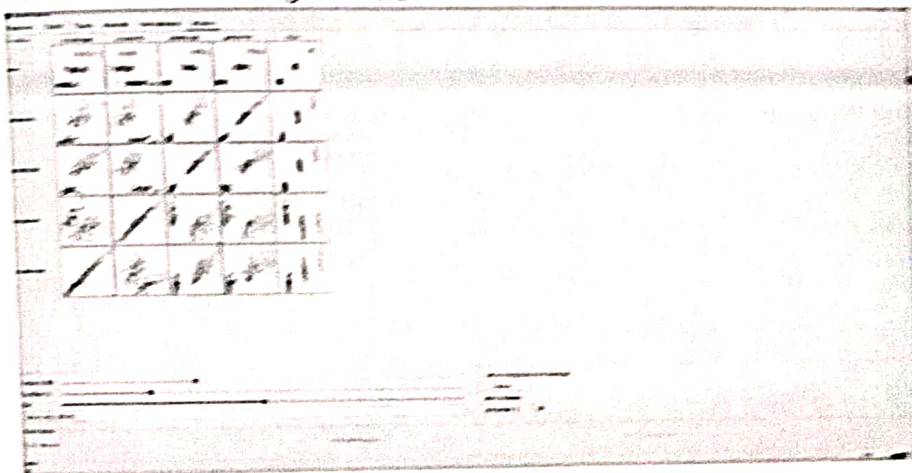
5. Change attribute combinations to observe scatter plots.



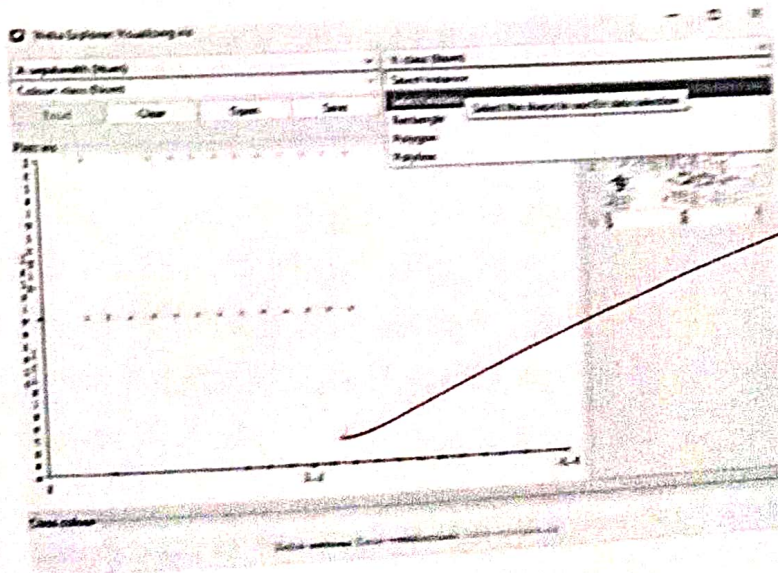
analyze relationships b/w attributes using multiple scatter plots.



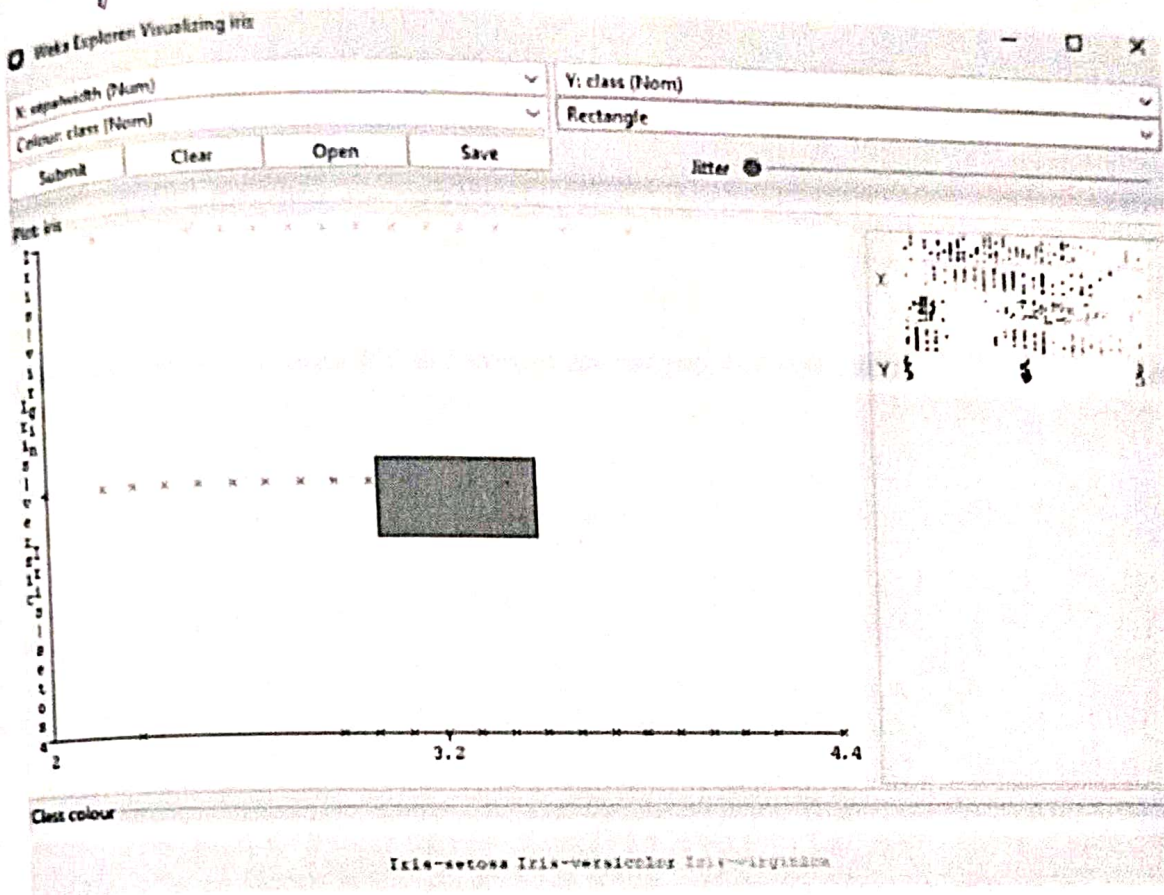
4. Use visualization panel to explore patterns & trends in dataset.



5. Select class attribute to color-code data points.



use selection tools to highlight specific data region.



LEARNING OUTCOME:

I learned to visualize datasets in WEKA and analyze relationships b/w attributes using scatter plot.

[Handwritten signature]

EXPERIMENT 8

PROBLEM STATEMENT: Implementation of Clustering technique on ARFF files using WEKA.

THEORY:

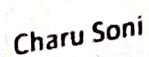
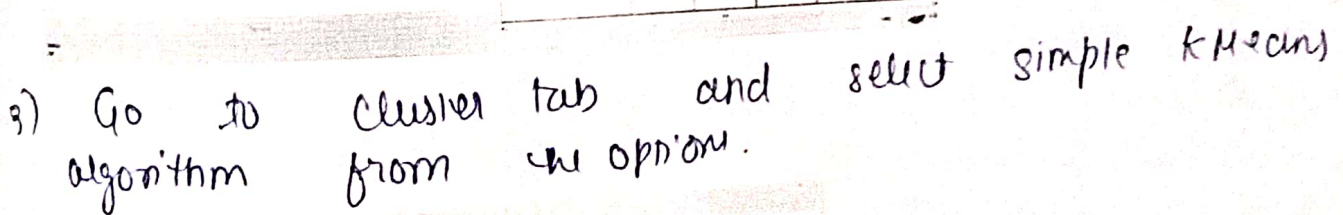
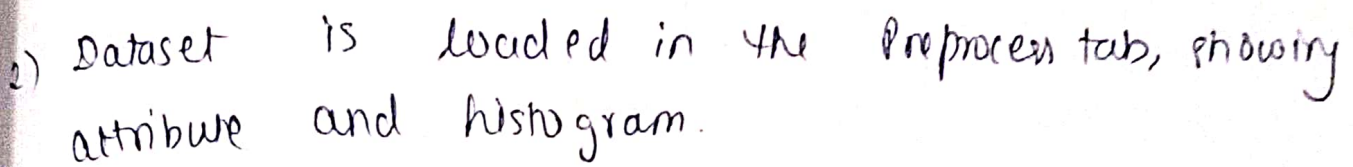
Clustering is an important data mining technique used to group similar data objects into clusters based on their characteristics. It is an unsupervised learning method, meaning it does not require labeled dataset and helps in discovering hidden patterns or structures within datasets.

In this experiment, clustering is implemented on ARFF files using Weka, a popular data mining tool. ARFF file contain both data and attribute information, making them suitable for analysis in Weka.

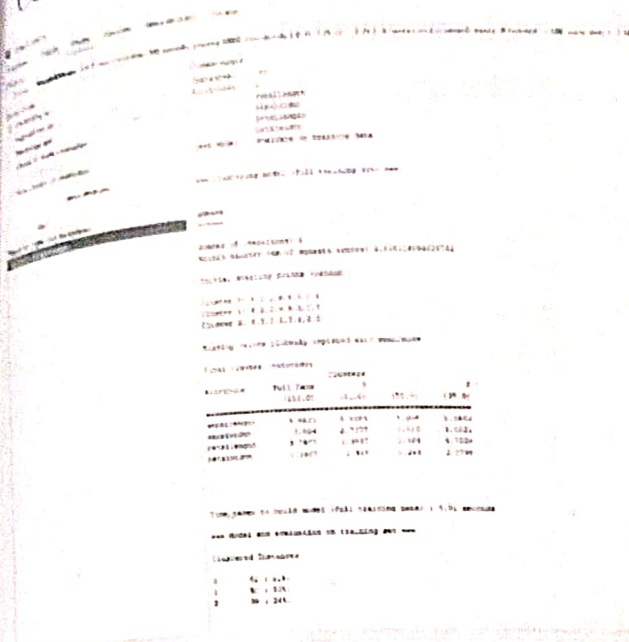
Weka provides various clustering algorithms such as K-Means and Hierarchical clustering. These algorithms group data into clusters based on similarity measures like distance. The clustering process involves selecting the dataset, choosing a suitable algorithm and analyzing the generated clusters.

Charu Soni

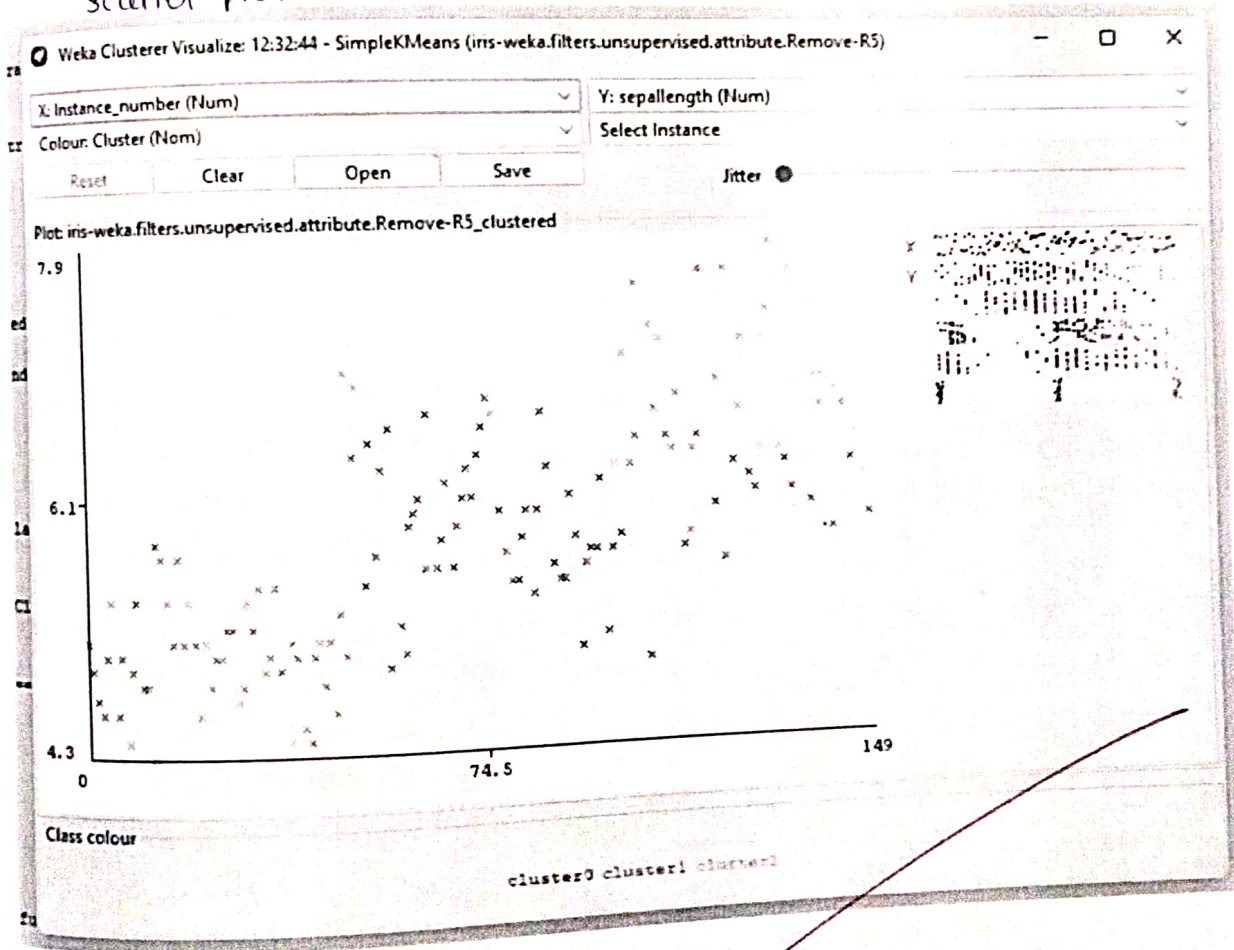
CODE:
1) Open Weka Explorer and click on open file to select an ARFF data set



click start to run clustering and view the generated output



Click Visualize Cluster Assignments to see clusters graphical scatter plot.



Learning Outcome:

I understood how to apply k-means clustering on ARFF dataset in WEKA.

Charu Soni

01417711623

EXPERIMENT 9

PROBLEM STATEMENT: Study of DBMINER tool.

THEORY:

DBMINER is a data mining tool used to extract knowledge such as associations, classifications and clusters from data cubes. (eg. foodMart cube). It provides a graphical interface to perform mining tasks without writing queries.

Logging In:

- Register and connect to an OLAP server.
- Access the foodMart data cube to perform mining operations.

① Install Required Software : Java or any other dependencies.

② Run the Installer : Double click the setup file to begin installation.

③ Follow Setup Wizard : Accept terms and proceed step-by-step.

④ Complete Installation : Install and wait.

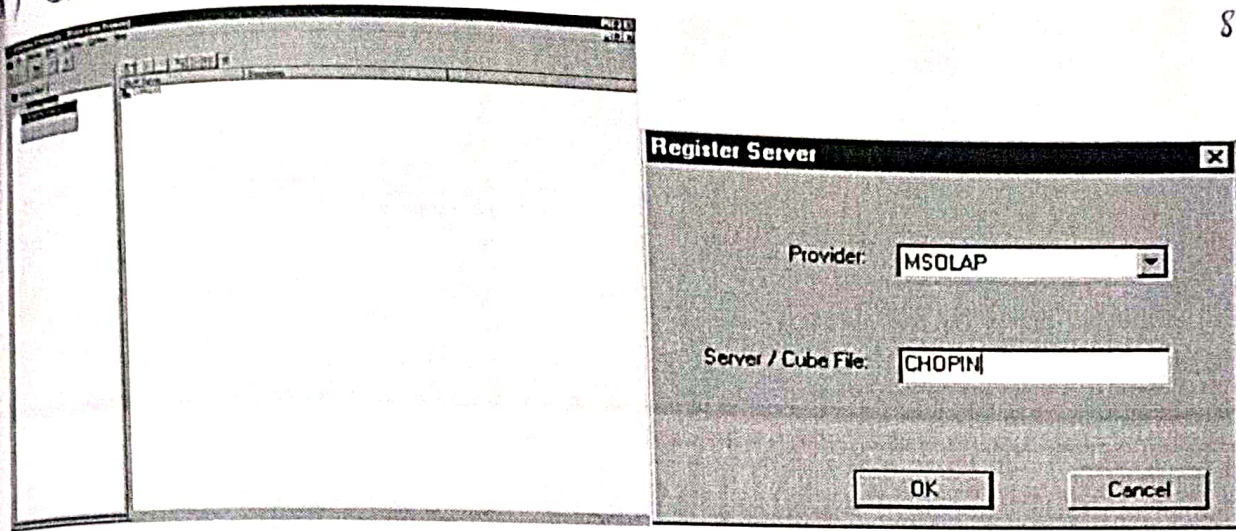
⑤ Launch DBMiner : Open tool from desktop.

⑥ Initial setup : Connect to a database to begin using DBMiner.

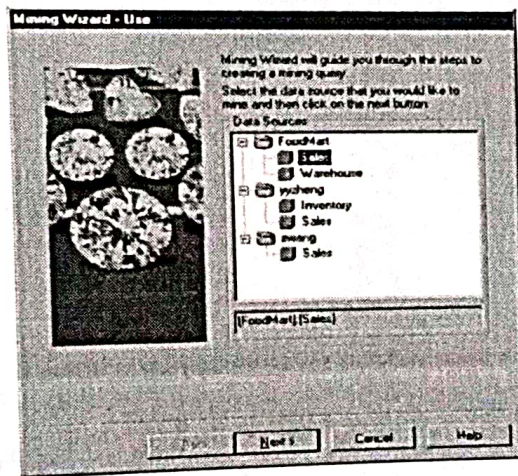
CODE:

1) Logging In (Register Server)

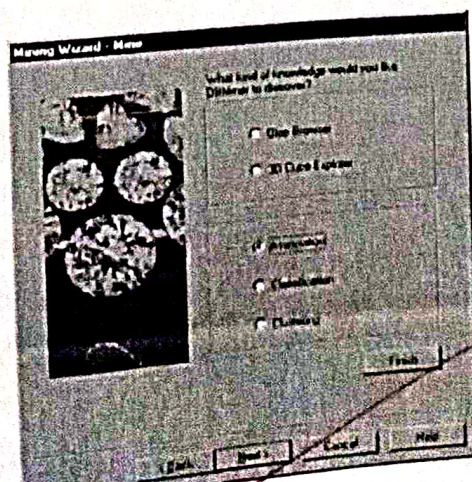
a) Start DBMiner and right-click on OLAP servers → Register new server.



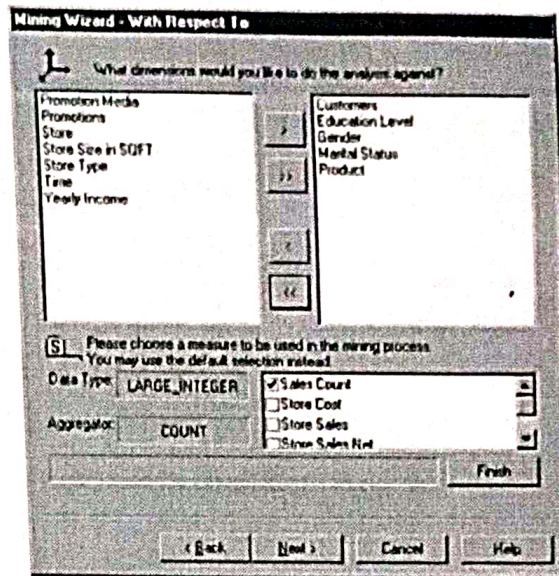
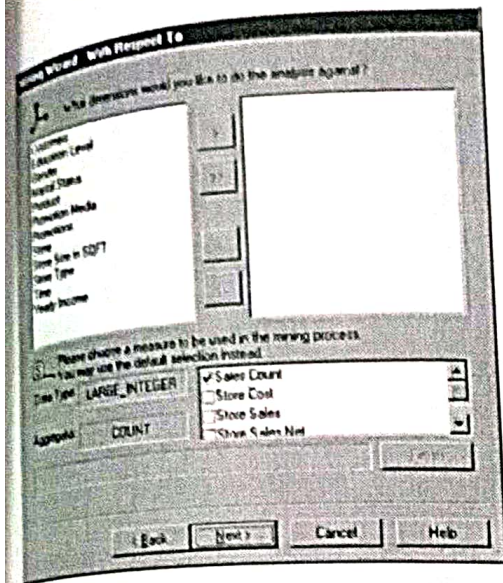
b) Enter server name and login.



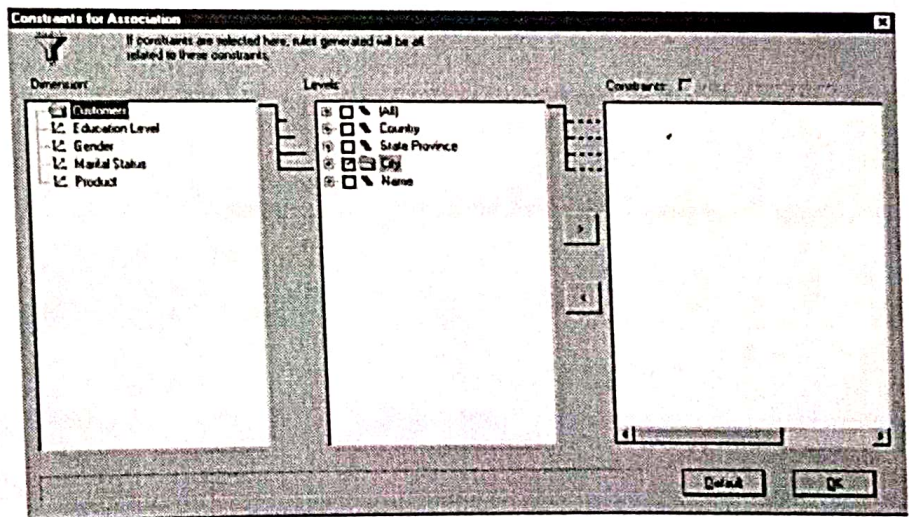
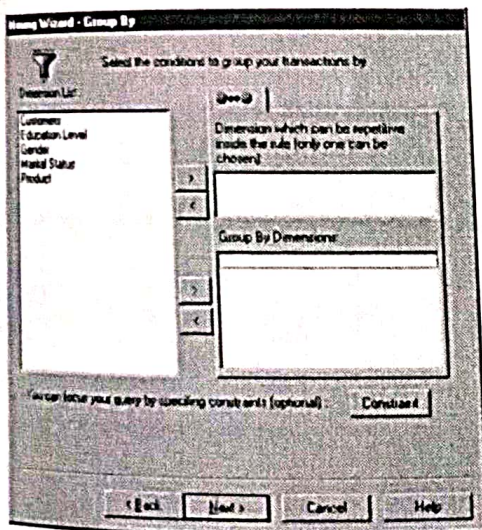
c) Access Food Mart data cube from OLAP Manager.



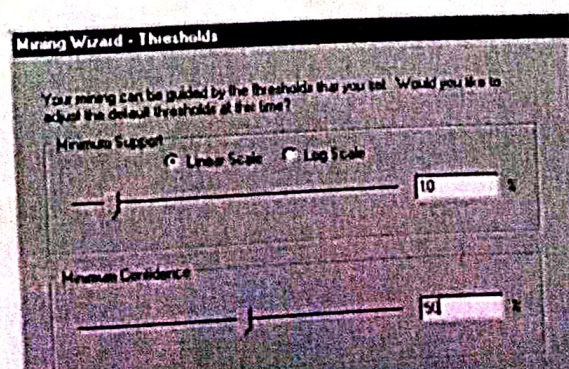
Go to mining → Mining Wizard and select data cube



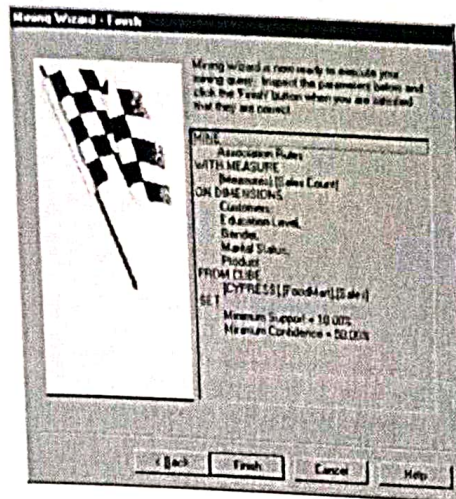
Choose Association as mining task



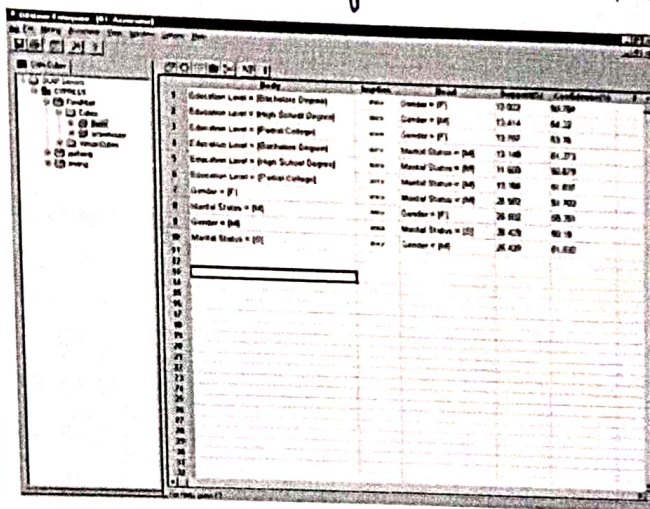
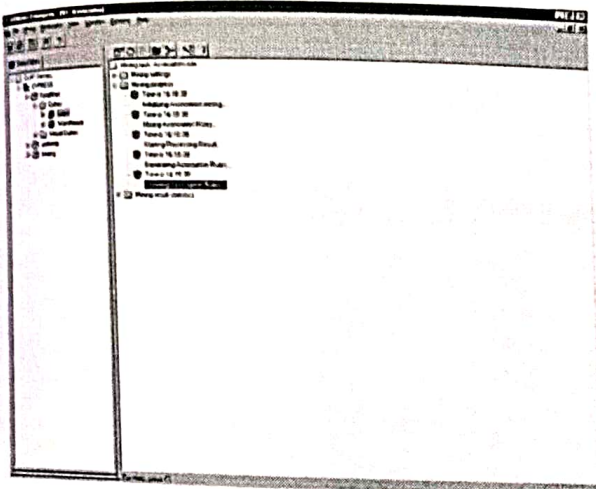
Select required dimensions.



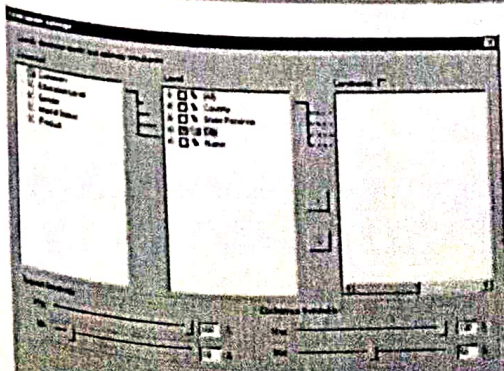
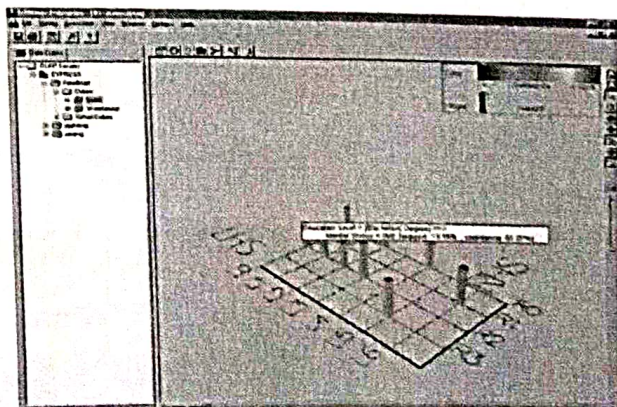
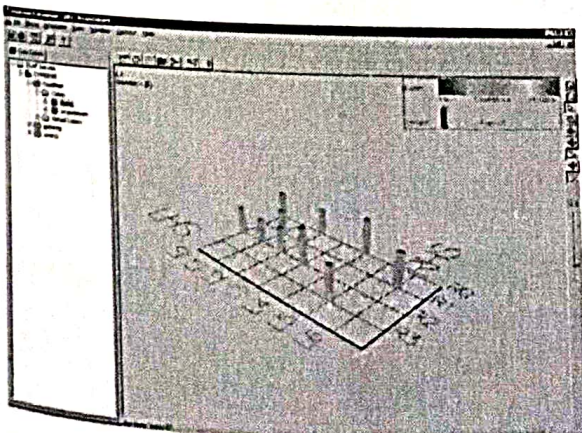
Choose association type.



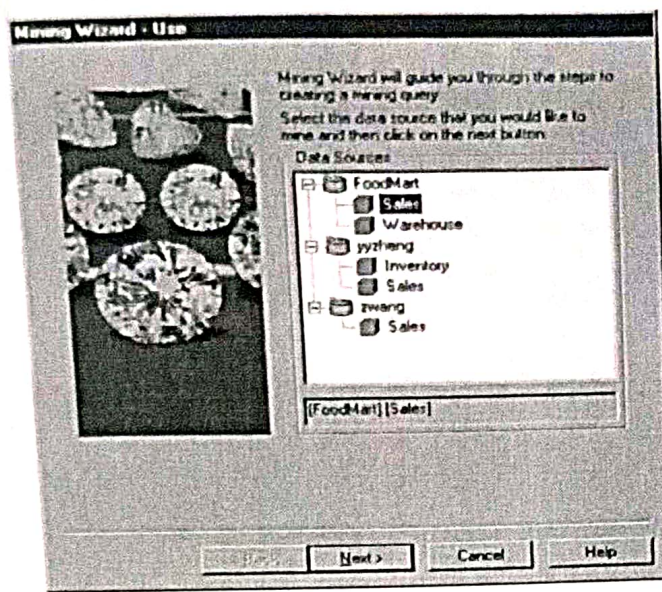
set minimum support 10% and confidence 50%.



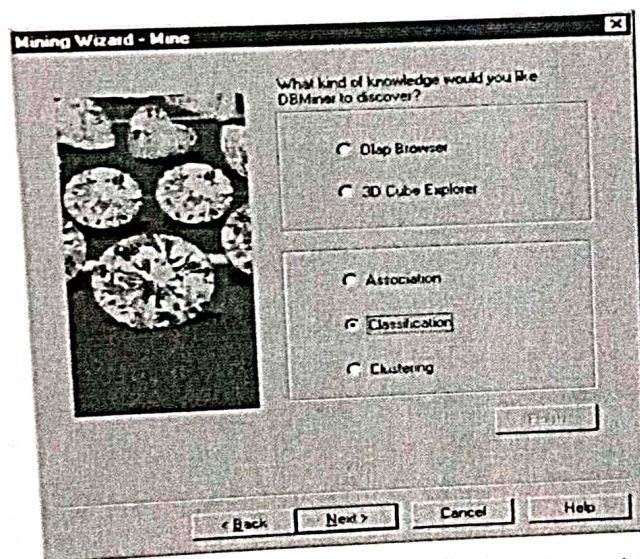
f) verify query and click finish.



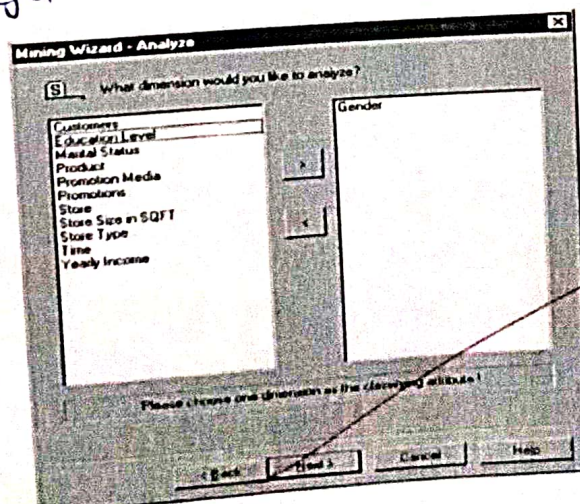
open Mining Wizard and select data cube



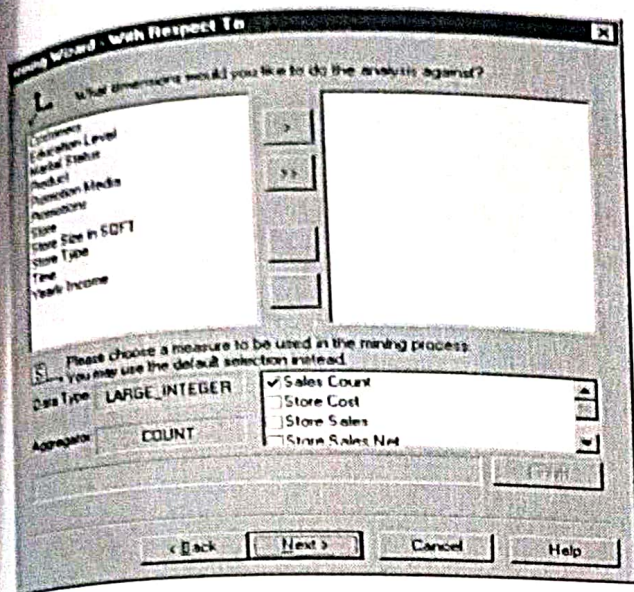
b) Choose Classification.



c) select target attribute. (eg Gender)



select related attributes.



Mining Wizard - With Respect To

What dimensions would you like to do the analysis against?

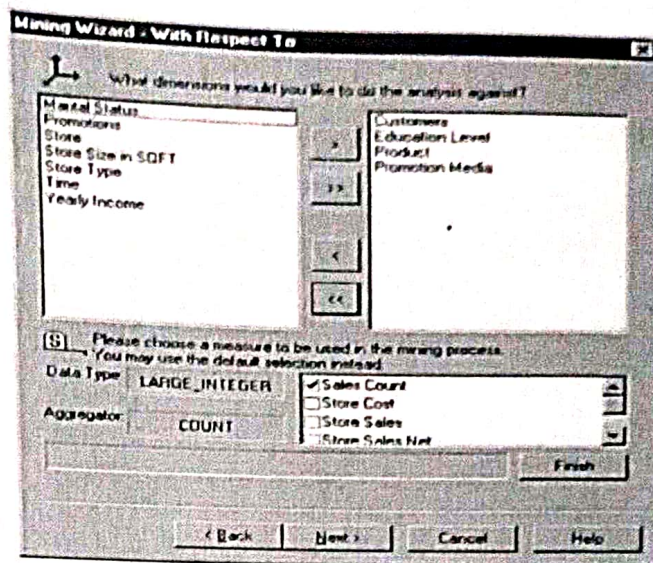
Customers
Education Level
Marital Status
Product
Promotion Media
Promotions
Store
Store Size in SQFT
Store Type
Time
Yearly Income

Please choose a measure to be used in the mining process.
You may use the default selection instead.

Data Type: LARGE_INTEGER
Aggregator: COUNT

☒ Sales Count
☐ Store Cost
☐ Store Sales
☐ Store Sales Net

< Back Next > Cancel Help



Mining Wizard - With Respect To

What dimensions would you like to do the analysis against?

Marital Status
Promotions
Store
Store Size in SQFT
Store Type
Time
Yearly Income

Customers
Education Level
Product
Promotion Media

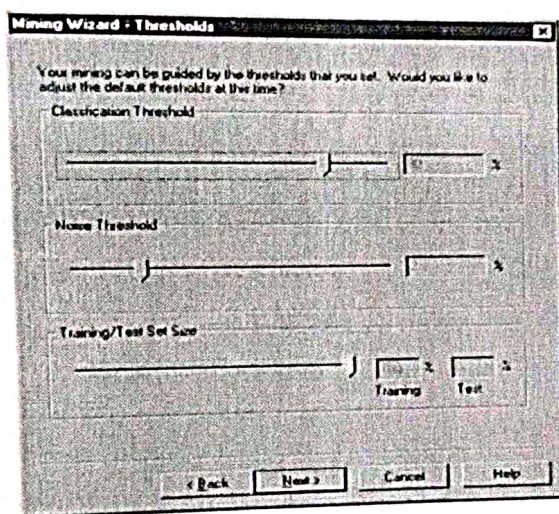
Please choose a measure to be used in the mining process.
You may use the default selection instead.

Data Type: LARGE_INTEGER
Aggregator: COUNT

☒ Sales Count
☐ Store Cost
☐ Store Sales
☐ Store Sales Net

< Back Next > Cancel Help Finish

e) Set thresholds



Mining Wizard - Thresholds

Your mining can be guided by the thresholds that you set. Would you like to adjust the default thresholds at the time?

Classification Threshold

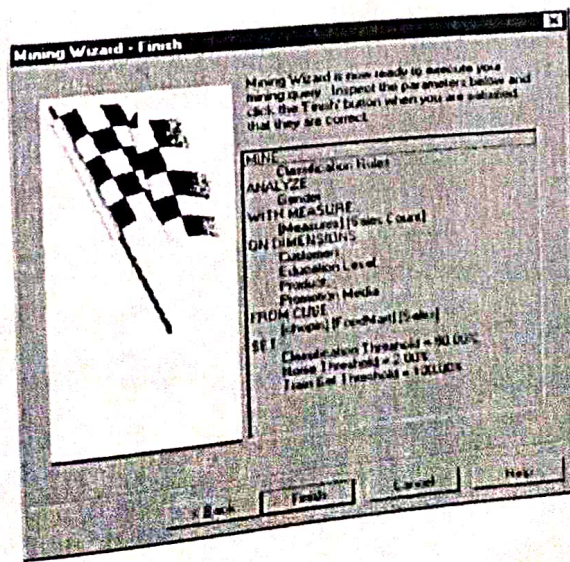
Noise Threshold

Training/Test Set Size

Training Test

< Back Next > Cancel Help

f) verify and click finish.



Mining Wizard - Finish

Mining Wizard is now ready to execute your mining query. Inspect the parameters below and click the Finish button when you are satisfied that they are correct.

MINING

Classification Model

ANALYZE

WITH MEASURE

ON DIMENSIONS

FROM CUBE

SET

Classification Threshold = 90.00%

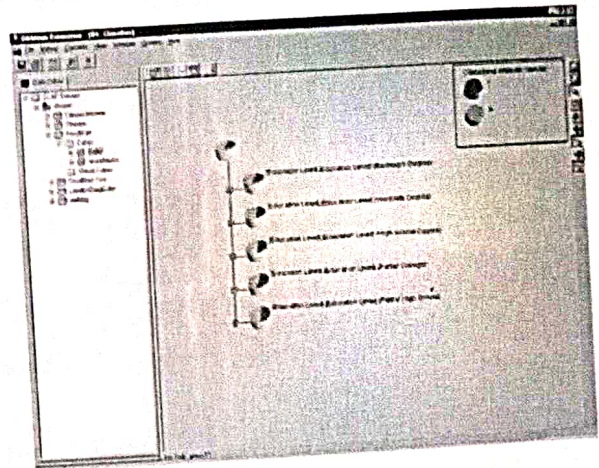
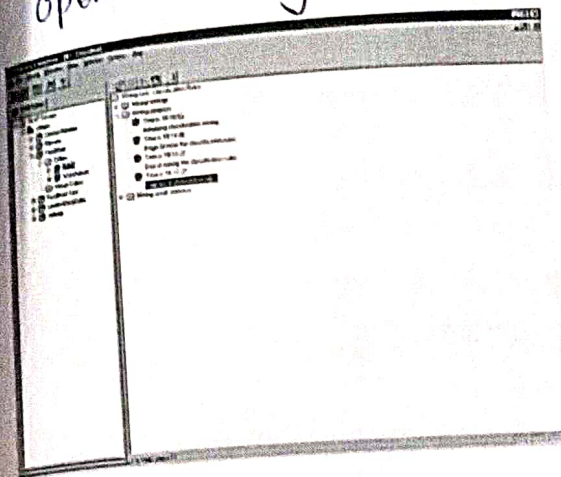
Noise Threshold = 2.00%

Train/Test Threshold = 100.00%

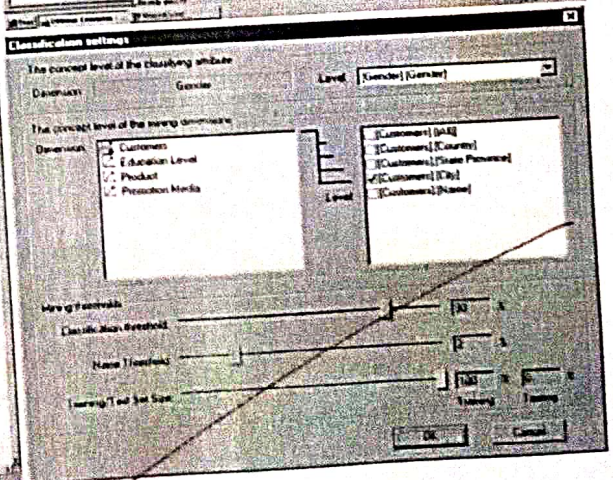
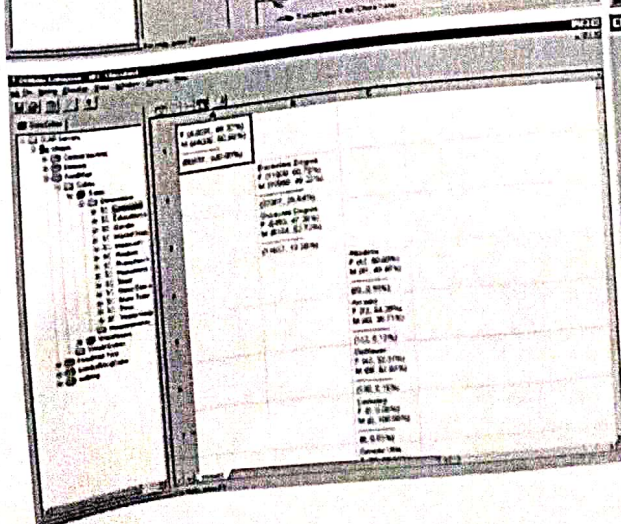
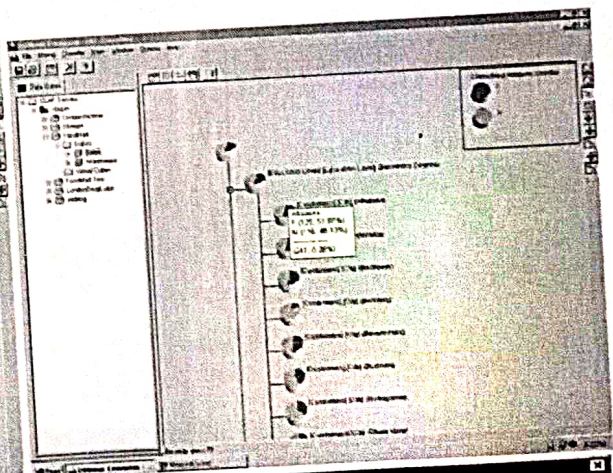
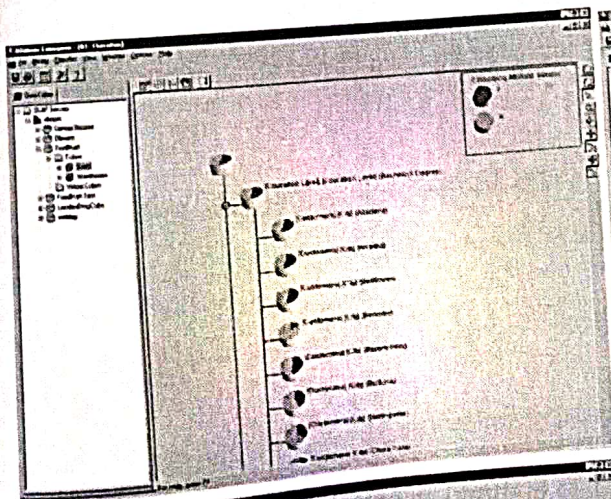
< Back Finish Cancel Help

mining clustering -

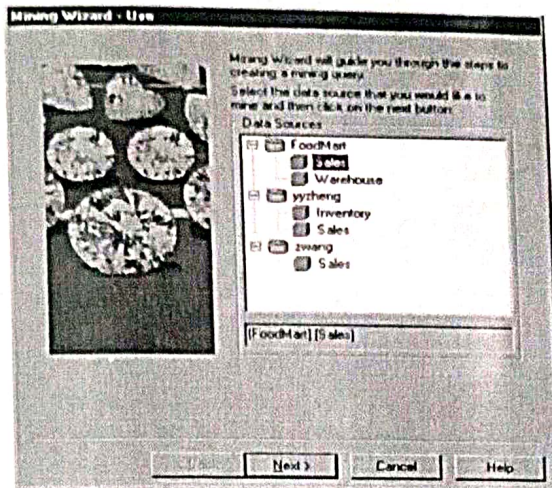
open mining wizard and select data cube.



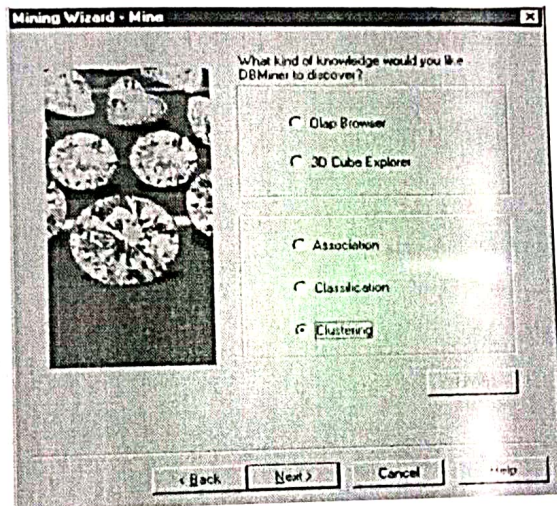
b) Choose clustering.



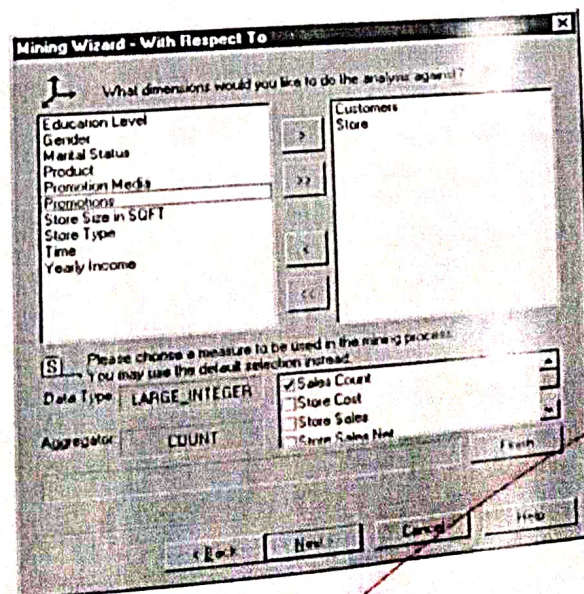
select two dimensions.



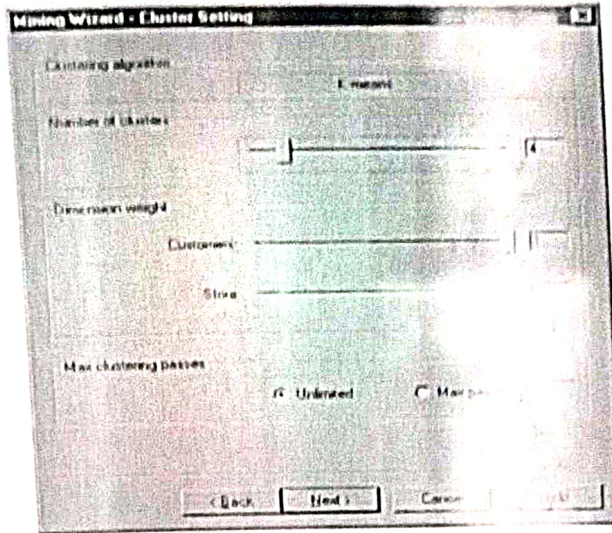
eg. Customer, Sales.



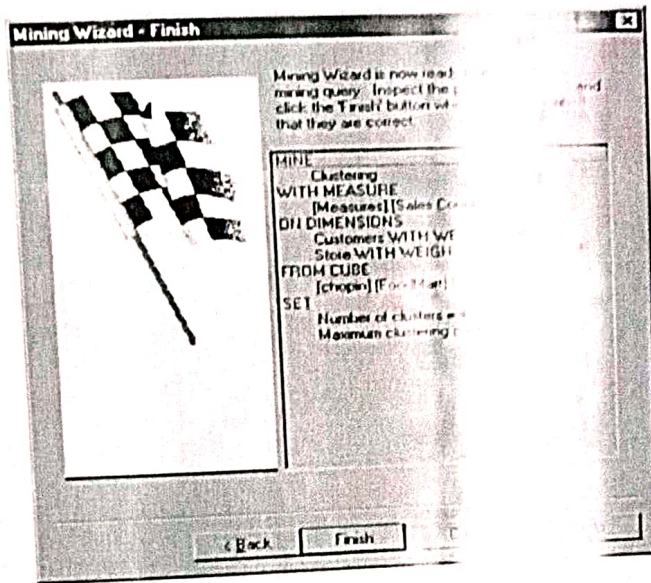
d) select parameter sales count



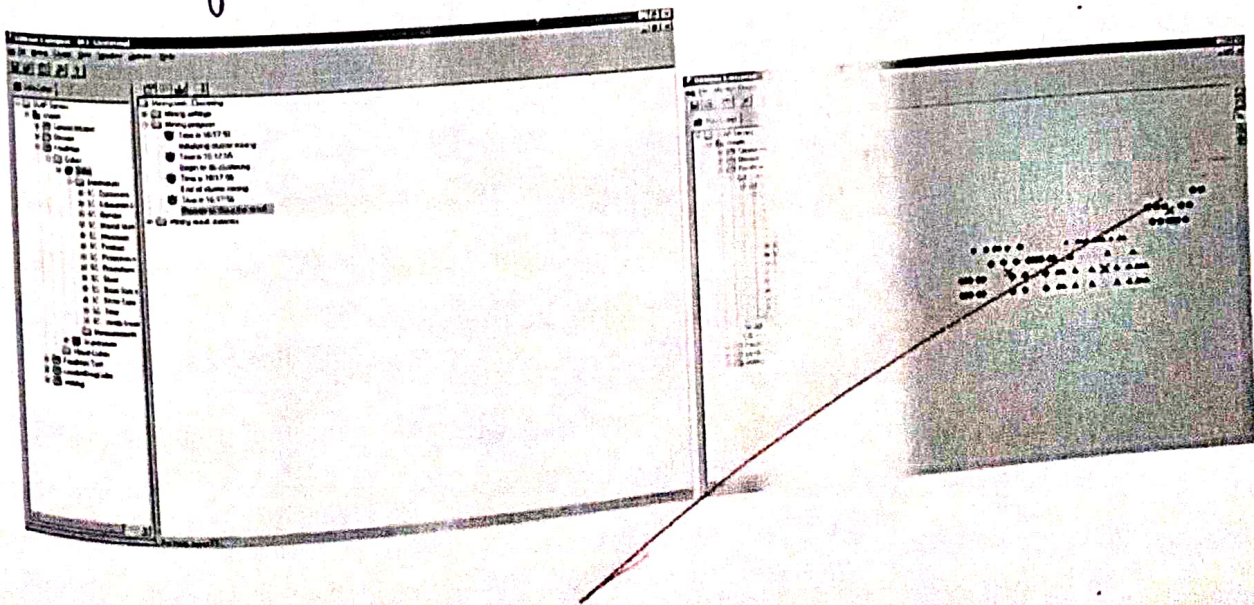
Set parameters (no. of clusters, weights, passes)



verify and click finish.



Result - clusters displayed in 2D graph and grid form.



EXPERIMENT 10

PROBLEM STATEMENT: Apply pre-processing and classification/regression techniques on a real-world dataset. Evaluate the performance of classification techniques using different parameters.

THEORY:

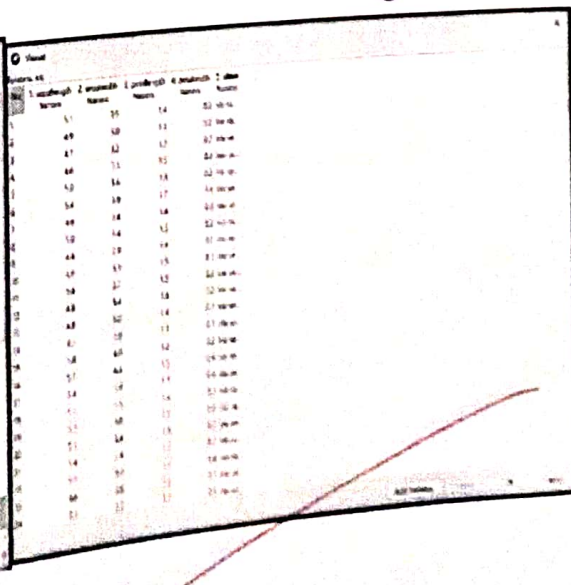
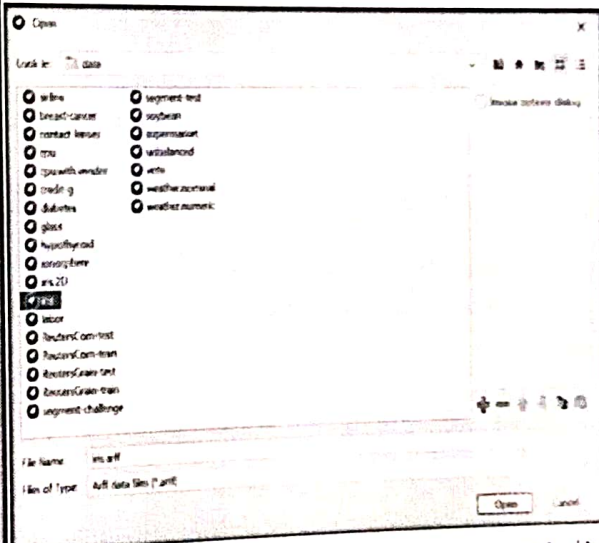
Real-world datasets are often noisy, incomplete and inconsistent, so data pre-processing is an important step. It includes data cleaning, data transformation and data reduction or feature selection. The dataset is then split into training and testing sets.

After pre-processing, classification techniques and regression techniques are applied to build predictive models. The model is trained on training data and tested on unseen data.

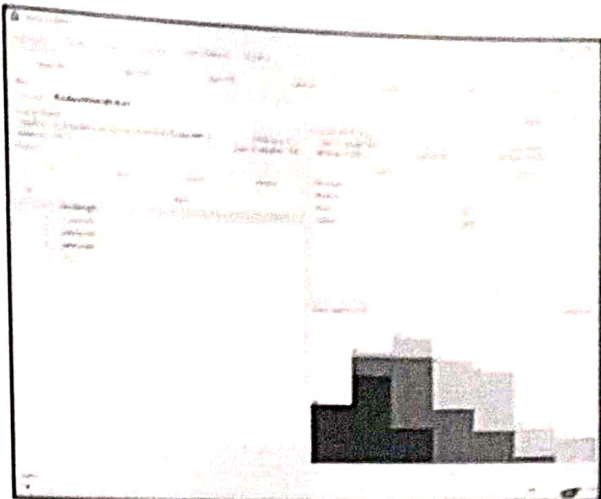
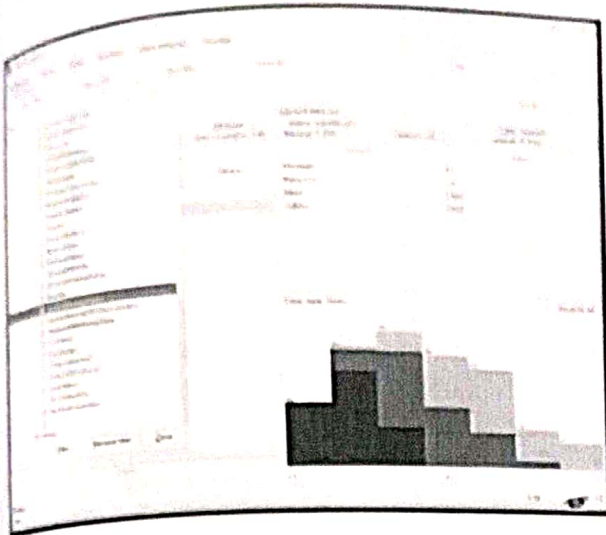
The performance of classification models is evaluated using parameters like accuracy, precision, recall and F1 score, which measure correctness and reliability. A confusion matrix is also used to visualize true or false predictions. These metrics help in comparing different models and selecting the best one.

[illegible]

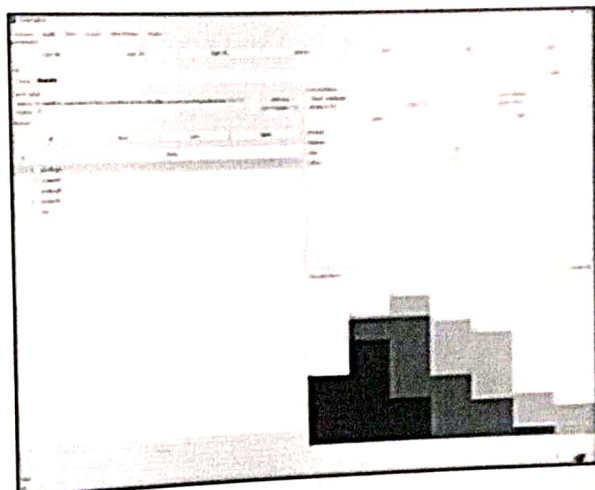
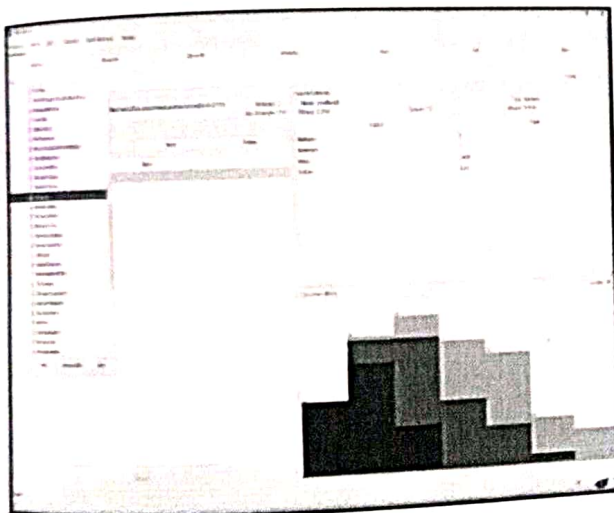
The screenshot shows a Windows Explorer window with the address bar set to 'data'. The left sidebar shows the 'data' folder selected. The main pane displays the contents of the 'data' folder, which include 'Local Disk (C:)', 'E:\MSN\Kil 1 (C:)', 'Network', 'Libraries', 'Gallery', 'Kunsh Software', 'Kunsh's S44 PE', and 'Downloads'. The 'File Name' field at the bottom shows '(Program Files)\Kunsh 1-5-4\data' and the 'File of Type' is set to 'All data files (*.*)'. The 'Open' button is highlighted.



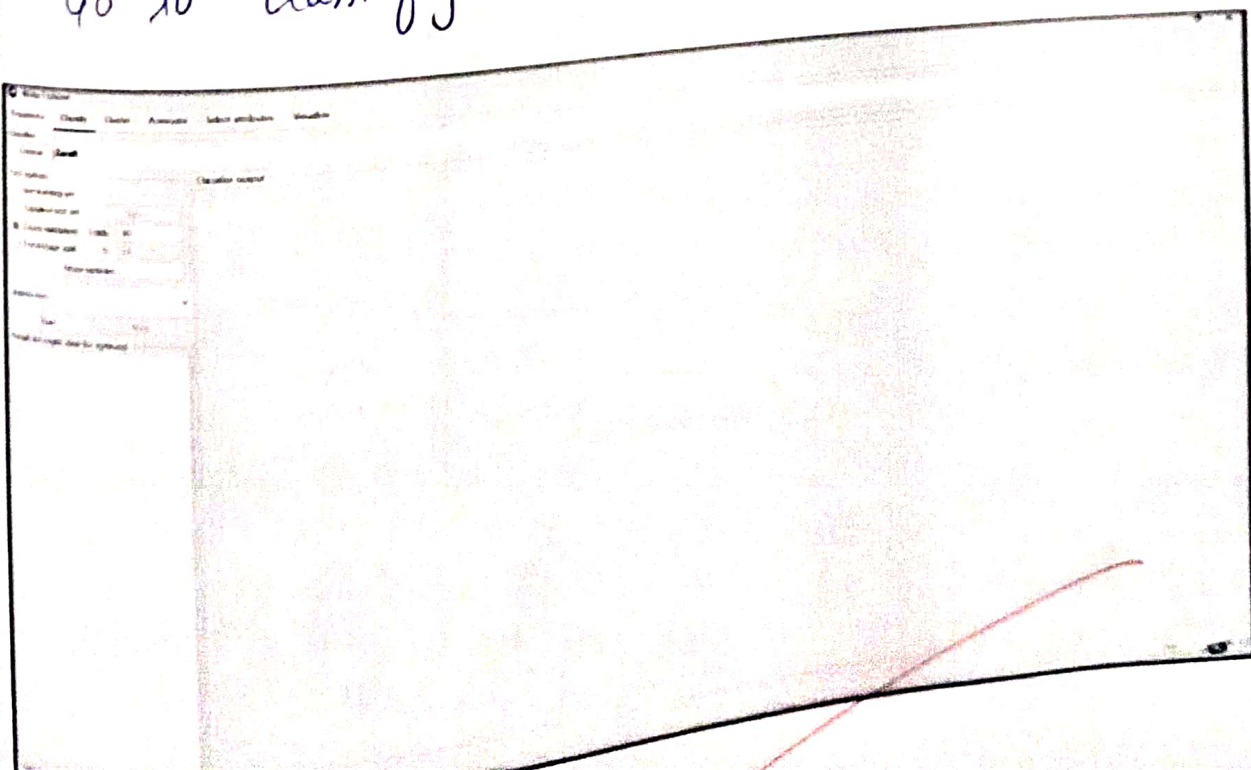
apply filters from filter option to prepared data.



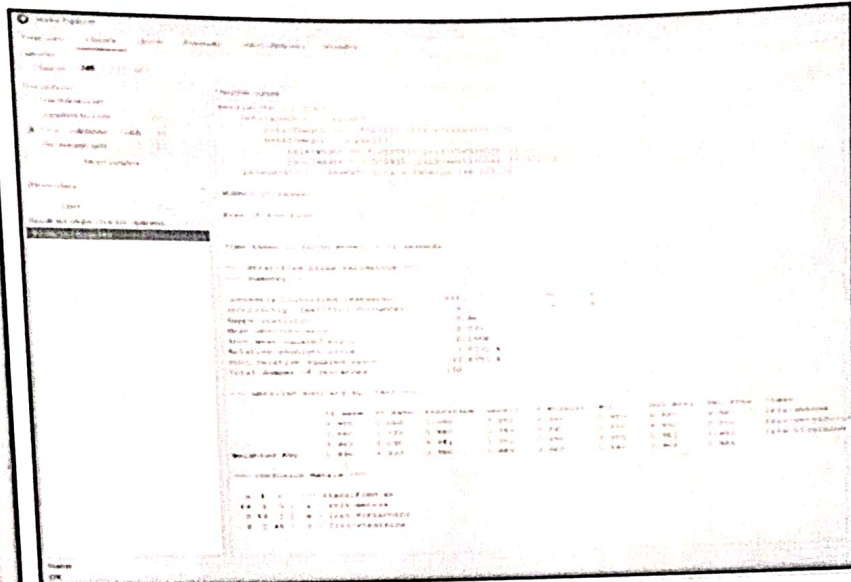
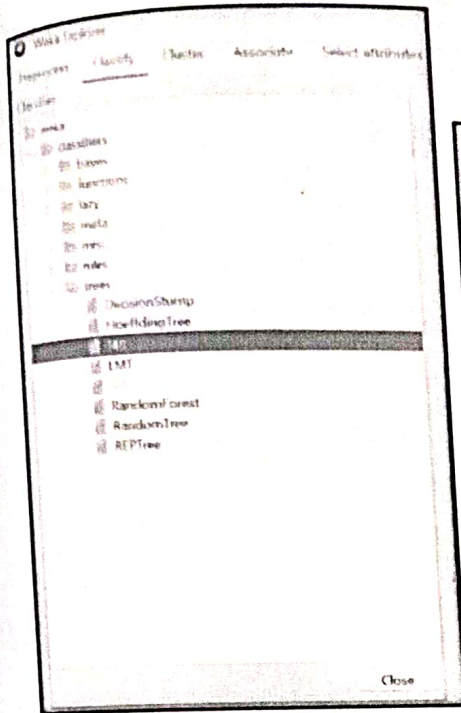
Observe updated dataset after preprocessing.



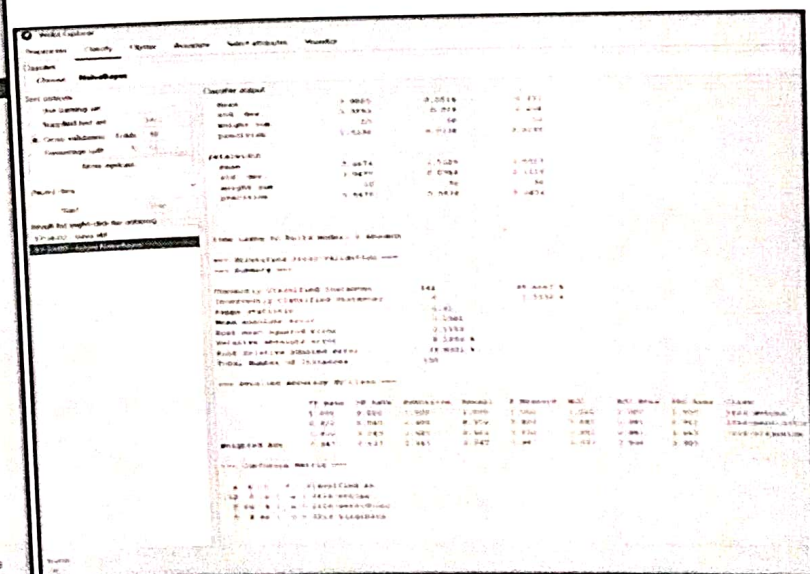
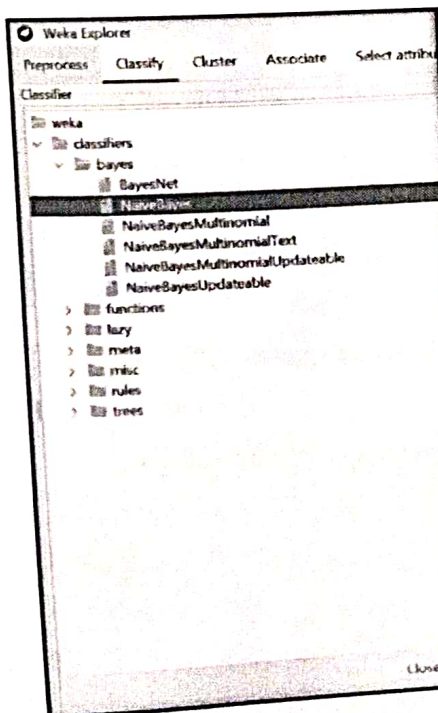
6. Go to classify tab to start classification process.



choose J48 classifier & click start to generate results.

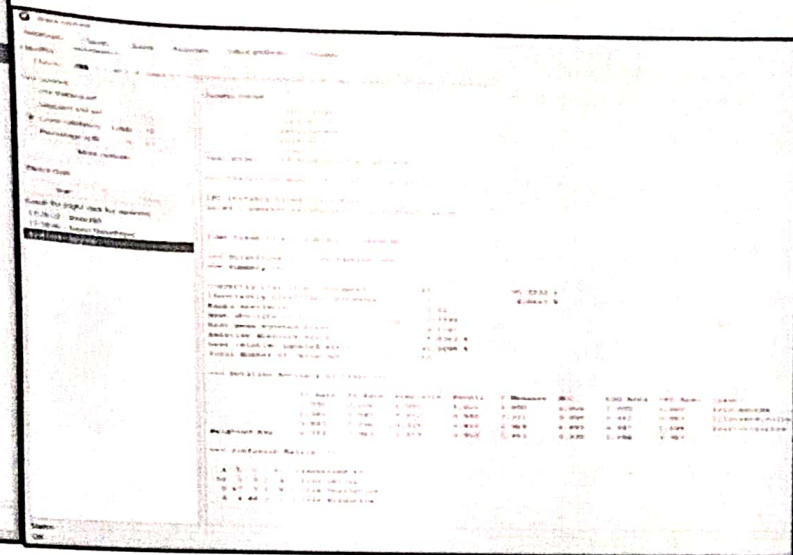
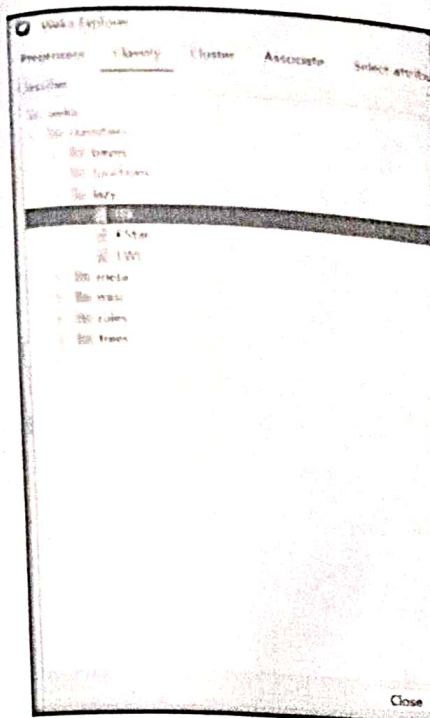


8. select Naive Bayes classifier and run to compare classification performance.

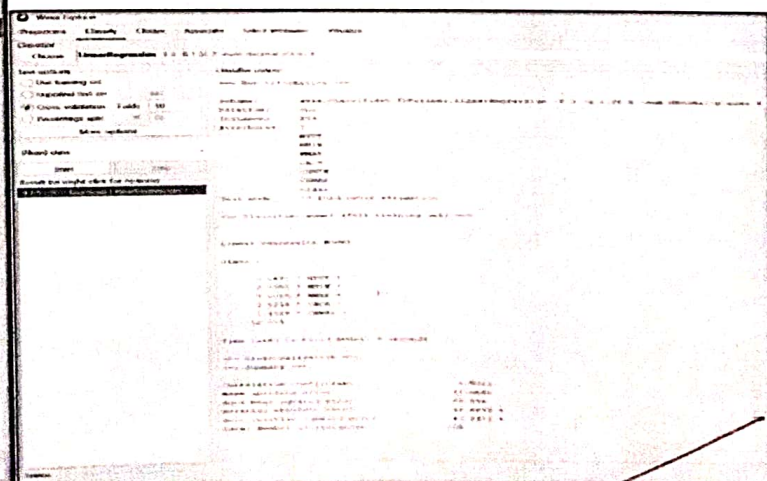
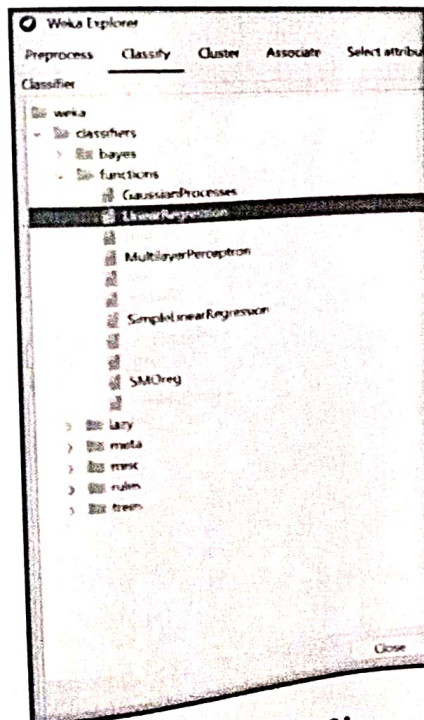


choose another results.

classifier (eg. J48 / lazy) & evaluate



10. Select Linear Regression under functions & click start to perform regression.



Learning Outcome:

I learned to preprocess real-world data & apply classification & regression techniques in WEKA.

Charu Soni

01417711623